Introduction

Information Security

Prof. Stefano Ferretti

Outline

- Part I: Few foundational terms and concepts
- Part II: Some use cases
- Part III: Lessons learned from use cases

Part I: Foundational Terms we Need

Can I give these terms for granted?

- Scalability
- Availability
- Single point of failure
- Bottleneck
- Integrity
- Confidentiality
- Privacy
- Fairness
- Auditability
- Client/Server model
- Peer-to-Peer model
- Churn

Maybe, we can focus on a subset

- Scalability
- Availability
- Single point of failure
- Bottleneck
- **Integrity**
- **Confidentiality**

- **Privacy**
- **Fairness**
- Auditability
- Client/Server model (**some issues only**)
- Peer-to-Peer model
- Churn → refers to the rate at which employees leave an organization. High information security churn causes a loss of institutional knowledge, disrupts security operations, and significantly increases an organization's vulnerability to breaches.

Integrity

Data integrity is the assurance that information remains accurate, complete, and consistent throughout its entire lifecycle

- Data not altered or destroyed in an unauthorized or accidental manner

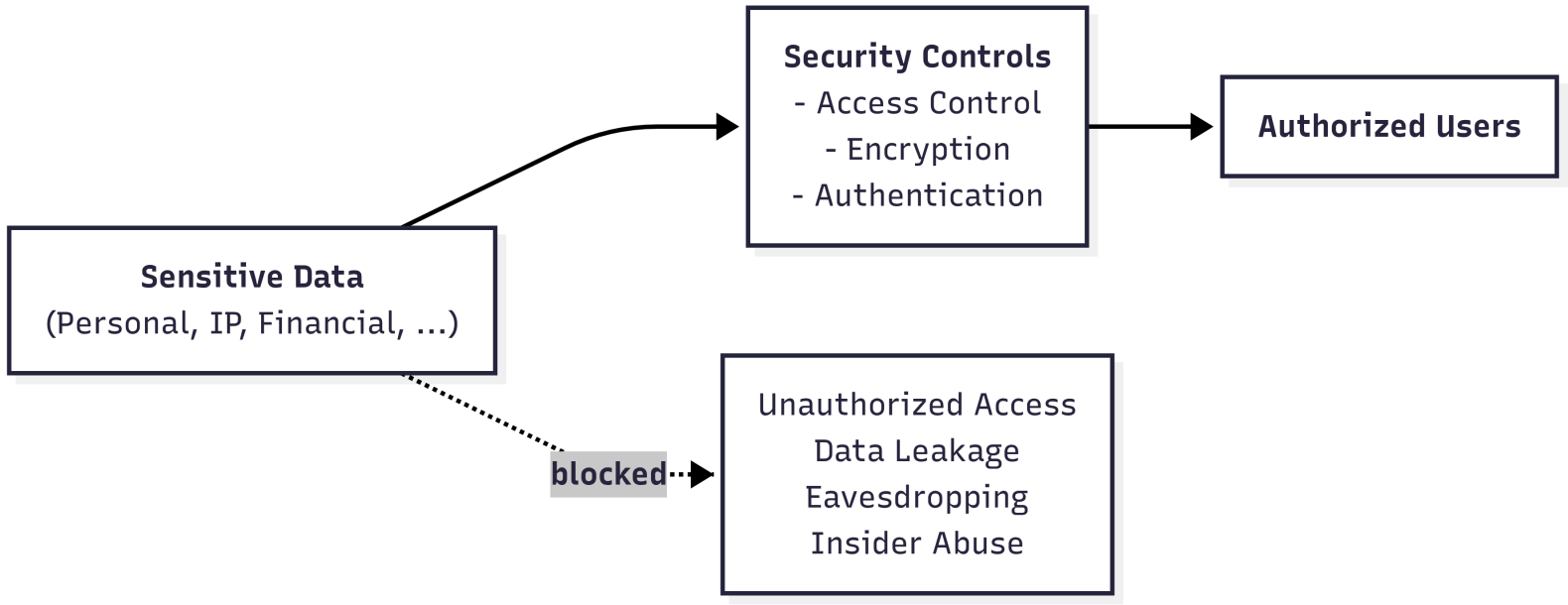
Integrity in ML

- **no data poisoning:** need to ensure training data is not altered
- **model integrity:** need to ensure model parameters are not altered

Confidentiality

Confidentiality ensures sensitive data is accessed only by authorized individuals

- Ensures that only authorized entities can access sensitive information
- Protects data from unauthorized disclosure or exposure



Privacy

Privacy is the right of individuals to exercise control over how their personal data is collected, processed, and shared, ensuring the confidentiality, integrity, and ethical use of their information

- **Privacy in Storage:** store only necessary data (**data minimization**), properly encrypted (**encryption at rest**)
- **Privacy in ML:** individuals not re-identified from model outputs (**membership inference protection**)

A Membership Inference Attack is a specific type of privacy attack in the field of machine learning.
- How the attack works: an attacker takes a record of an individual (e.g. Mario Rossi) and feeds it into the model. The attacker analyses the model's response (often by looking at the prediction confidence scores). Since models tend to respond much more accurately to data they have already seen during training than to entirely new data, the attacker can determine with a very high degree of certainty whether Mario Rossi's data was part of the training dataset.
- Why is this serious? Knowing that an individual is part of the dataset can, in itself, constitute a very serious privacy violation. For example: if the model in question was trained exclusively on patients with a specific rare disease, discovering that Mario Rossi's data was used for training automatically reveals that Mario Rossi has that disease. Protecting against this attack means ensuring that the model's behaviour is statistically identical whether an individual's data was included in the training or not.

Privacy vs Confidentiality

- **Confidentiality $\rightarrow$ access control**
- **Privacy $\rightarrow$ inference control** → Inference control analyzes and restricts the information that a user can deduce (infer) by querying a system or observing its outputs.

Fairness

Fairness ensures that information systems and algorithms operate without creating or reinforcing bias. It implies the equitable (fair) treatment of all users and data subjects, preventing discriminatory outcomes and ensuring balanced resource allocation across the network

- **Fairness in distributed systems**: fair resource allocation, laod balancing, latency equity
- **Fairness in ML**: bias mitigation, group fairness, algorithmic transparency

The problem of trust

Central Trusted Authority

In the traditional model of computer science (e.g., the classic client-server paradigm), security relied on the presence of a central entity that everyone trusted implicitly (the Central Trusted Authority). Examples: A bank's central server or a hospital's central database.

What we **trust**

- **services**
- **data sources**
- **data**

However, in modern systems, **trust cannot be assumed**

Two Fundamental Objectives

Systems should be analyzed along two axes (~~**besides**~~ in addition to **performance**):

- **Trustworthiness**
  → Can we rely on data, models, and system behavior?

In modern systems, ensuring trustworthiness means implementing mechanisms to ensure that:
- The data is intact: No one has altered it at the source (preventing attacks such as data poisoning).
- The models are robust: The algorithm makes correct decisions and is not easily fooled by malicious inputs.
- The behavior is predictable: The system does exactly what is expected.

- **Privacy**
  → Can we use data without exposing sensitive information?

Modern systems have a hungry need for data to function properly (the more clean data there is, the more AI learns). The Privacy axis focuses on ethical and legal constraints: how can we extract value, statistics, and patterns from this massive amount of data without ever violating the anonymity or rights of the individuals who generated that data?

These are **not independent** and often introduce trade-offs

Trust vs Privacy: A Tension

Improving one objective may weaken the other

Examples:

1
- Logging improves **auditability** but may reduce **privacy**
- Data sharing improves **utility** but increases **exposure risk**

2
- Differential Privacy improves privacy but reduces model accuracy

1) The Control Trade-off (Verification vs. Anonymity) to increase Trustworthiness: If I want to be 100% sure that a data source is reliable and isn't sending false data (e.g., fake reviews, altered GPS coordinates), the most logical approach is to identify the data sender. Knowing exactly who sent what destroys the user's privacy. Conversely, if I guarantee total anonymity to protect privacy, I open the door to malicious actors who can send junk data without being detected, undermining the system's trustworthiness.

2) The Trade-off of Statistical Noise (Differential Privacy vs. Accuracy) to Increase Privacy: To protect ML models from privacy-violating attacks (such as membership inference), we use Differential Privacy, which introduces "random noise" into the data or the model's calculations. If I introduce too much noise, the model becomes less accurate, less stable, and potentially less robust. If a medical model misdiagnoses a patient because we introduced too much noise to protect patient privacy, the system is no longer trustworthy.

Does the AI revolution change the overall picture?

In terms of trustworthiness, privacy, scalability, fairness, auditability, etc.

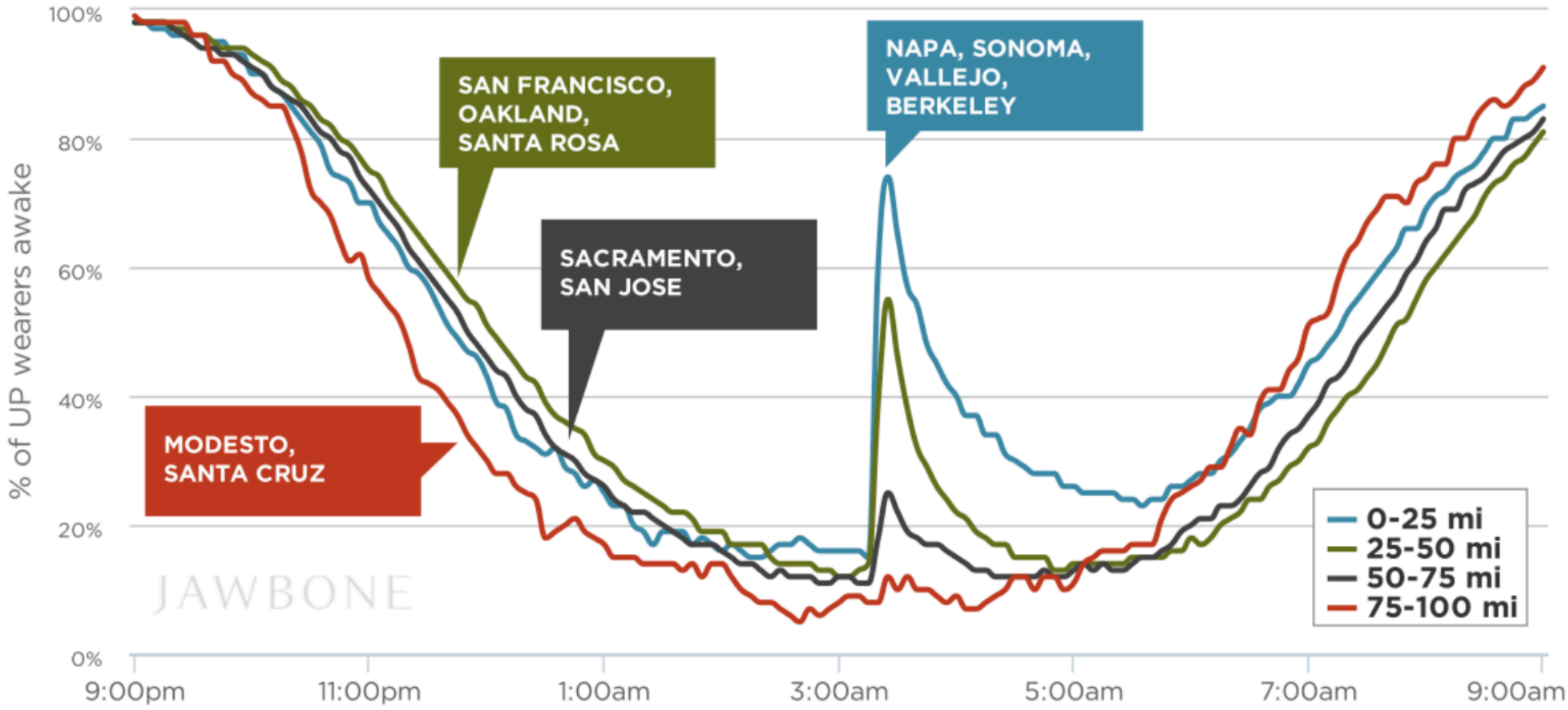
Does the AI revolution change the overall picture?

In terms of trustworthiness, privacy, scalability, fairness, auditability, etc.

Trustworthy AI

Designing and ensuring Trustworthy AI means creating Artificial Intelligence systems that humanity can rely on safely and ethically, even in the presence of hostile environments or malicious attackers.

The landscape has changed



The more data you collect from various sources, the more your knowledge grows. By combining different data, algorithms can identify stable correlations and recurring patterns that were previously invisible. From these correlations, deep insights, predictive models, and strategic decisions are derived.

The landscape has changed

The more data is publicly available, the larger the attack surface becomes, and the more people (or organizations) are exposed.
Inference: An attacker can use seemingly harmless pieces of information and, by combining them, deduce (infer) a secret that was never publicly disclosed.
Re-identification: Even if a company publishes an anonymized dataset, an attacker can take that dataset and cross-reference it with other OSINT sources. Through this correlation, the attacker can uncover the real identity behind that anonymous data.

Open Source Intelligence (OSINT)

More data → More knowledge

- Correlation
- Patterns
- Insights

Aggregation is power

More data → More exposure

- Inference
- Re-identification
- Hidden sensitivity

Aggregation is risk

Real-world example

Netflix Prize Dataset (2006)

- “Anonymized” movie ratings released
- No names, no direct identifiers

↓

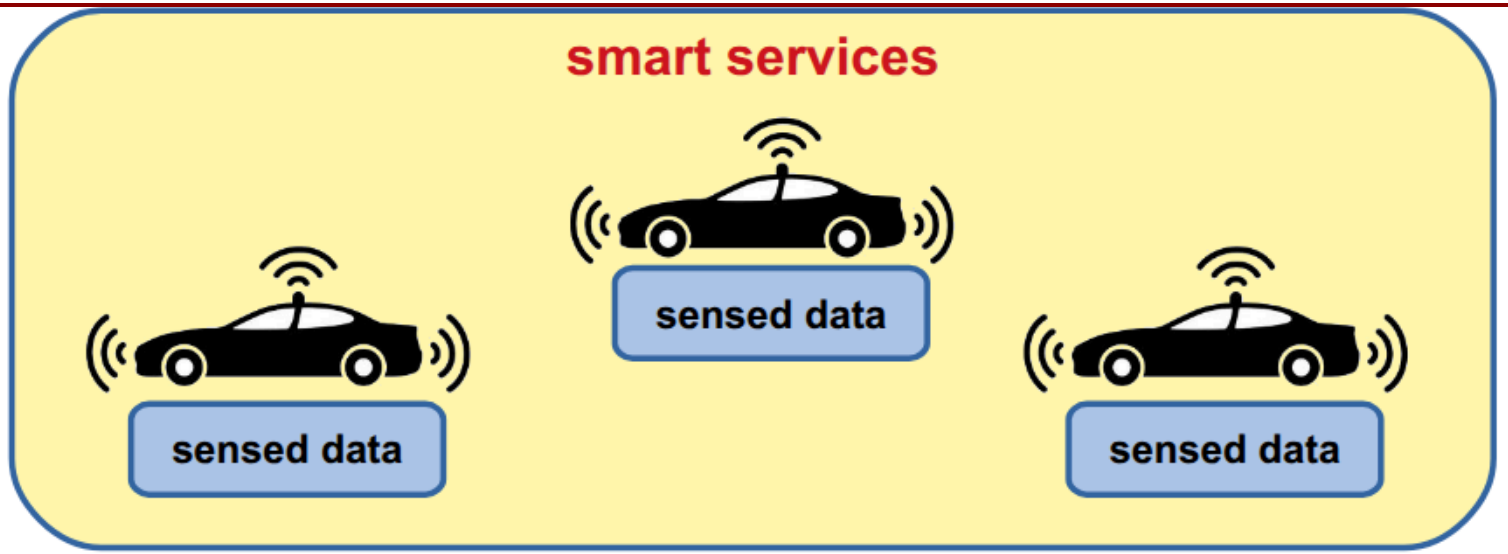
- Cross-referenced with IMDb ratings → The Netflix dataset was cross-referenced with public data extracted from IMDb (Internet Movie Database), a well-known online platform (and source of OSINT) where users review and rate movies in a completely public manner, signing in with their first and last names or a traceable nickname.
- Users re-identified

Part II: Some Use Cases

Use Case #1: Crowdsensing Applications

(in Decentralized Systems)

Smart Transportation Systems



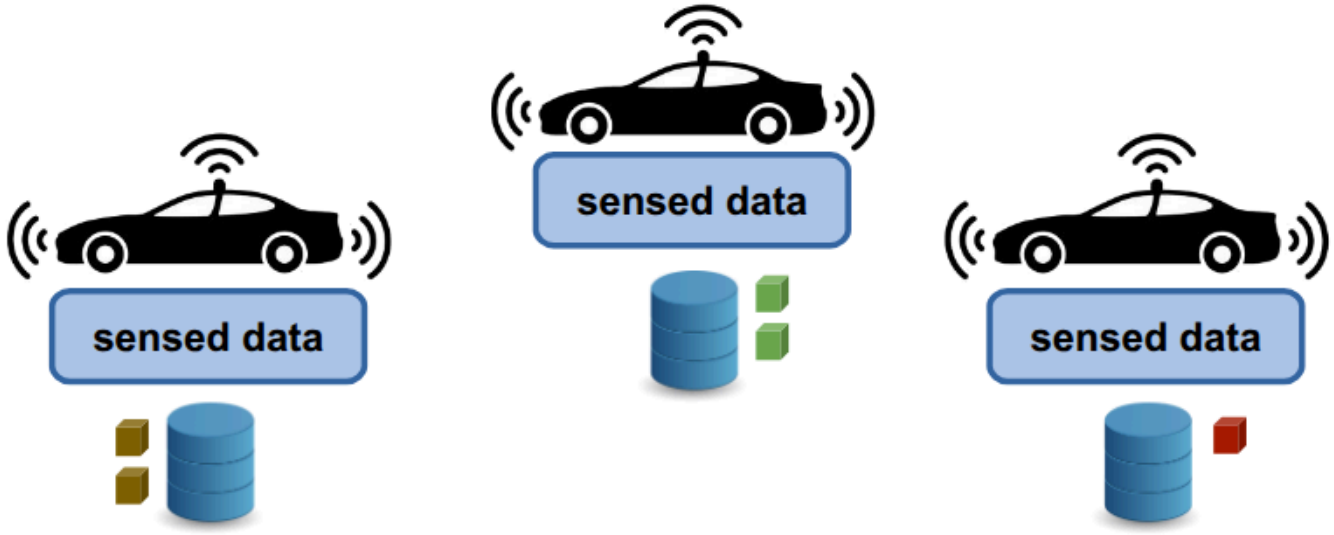
Smart Transportation Systems

The system implements **crowdsensing-as-a-Service** → Crowdsensing is a paradigm in which a multitude of ordinary users, using sensors built into their everyday mobile devices (such as smartphones, smartwatches, or in-car computers equipped with GPS, accelerometers, cameras, etc.), collect and share data useful for monitoring a phenomenon of common interest. When this paradigm becomes as-a-Service, it means that a decentralized technological infrastructure is established to continuously collect, aggregate, and process this mass data in order to resell it or offer it in the form of application services to users or companies.

Data used to provide:

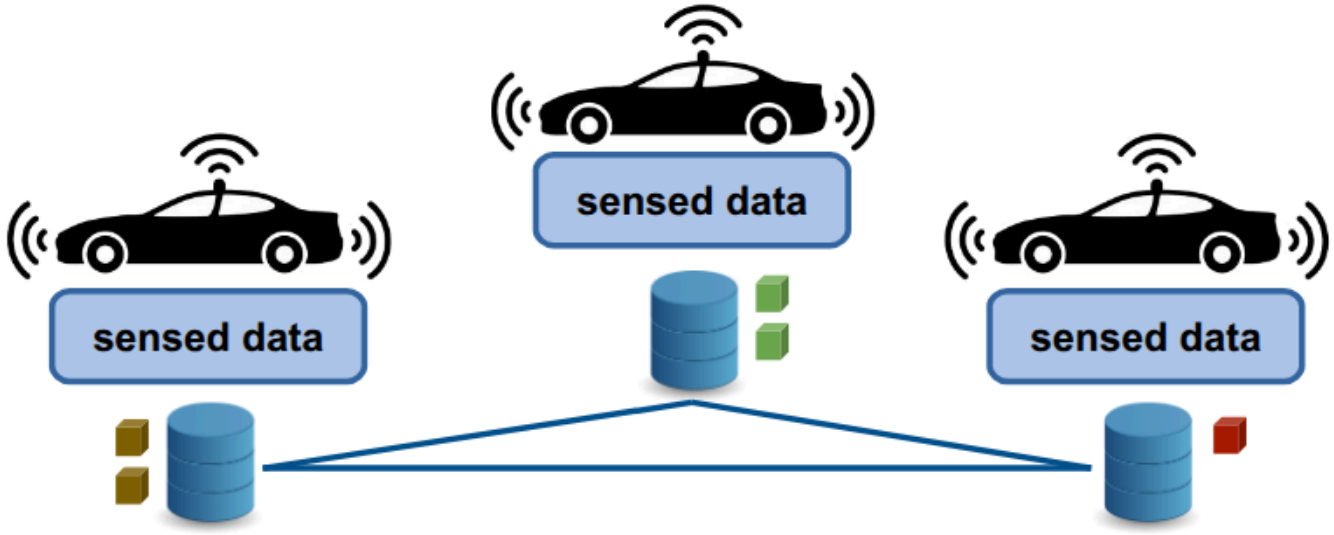
- Traffic analysis
- Route optimization
- Safety alerts (e.g., crash, wrong-way driving, weather conditions)

Smart Transportation Systems



Personal data

Smart Transportation Systems



Data storage

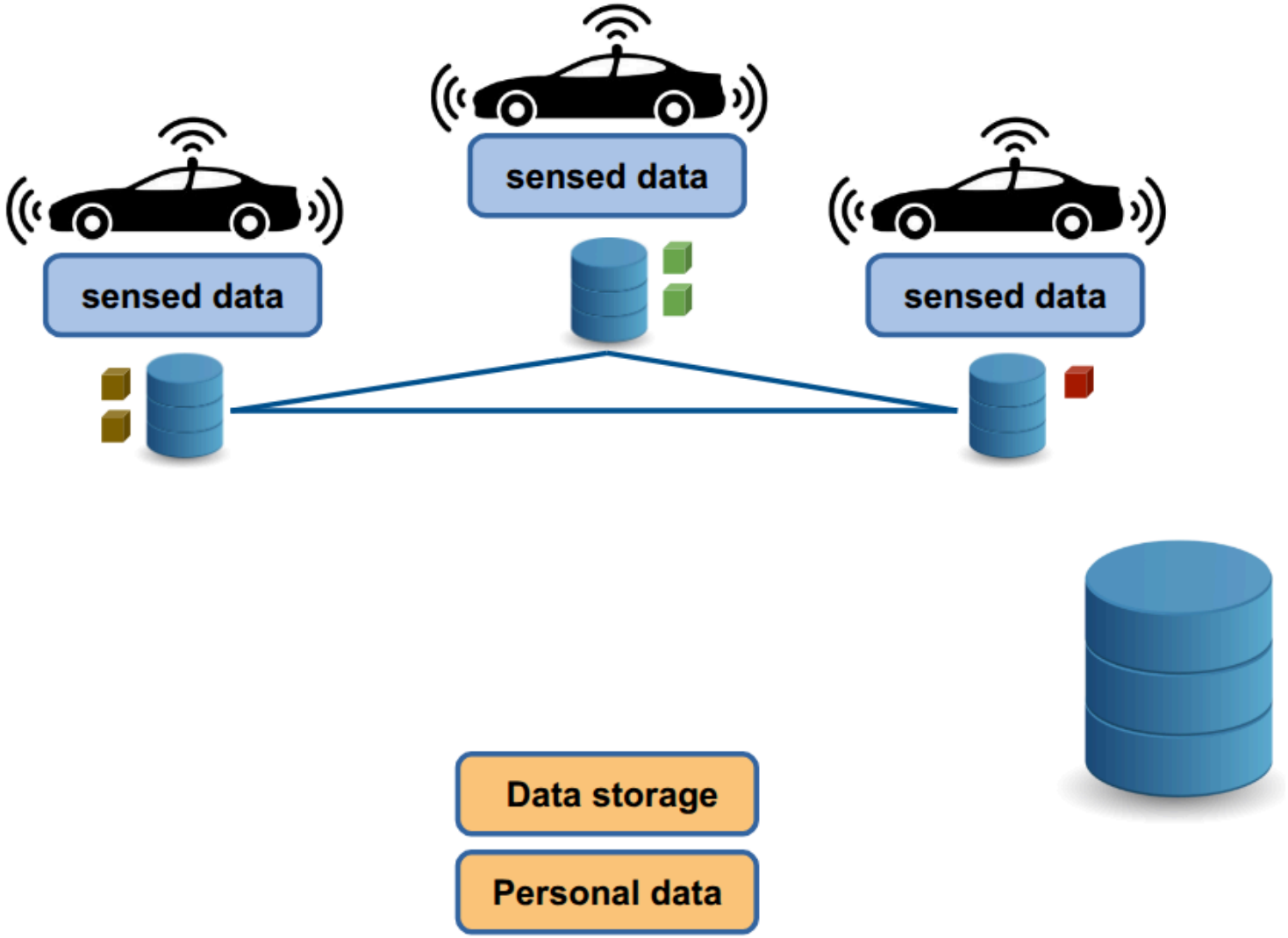
Personal data

Security Objectives

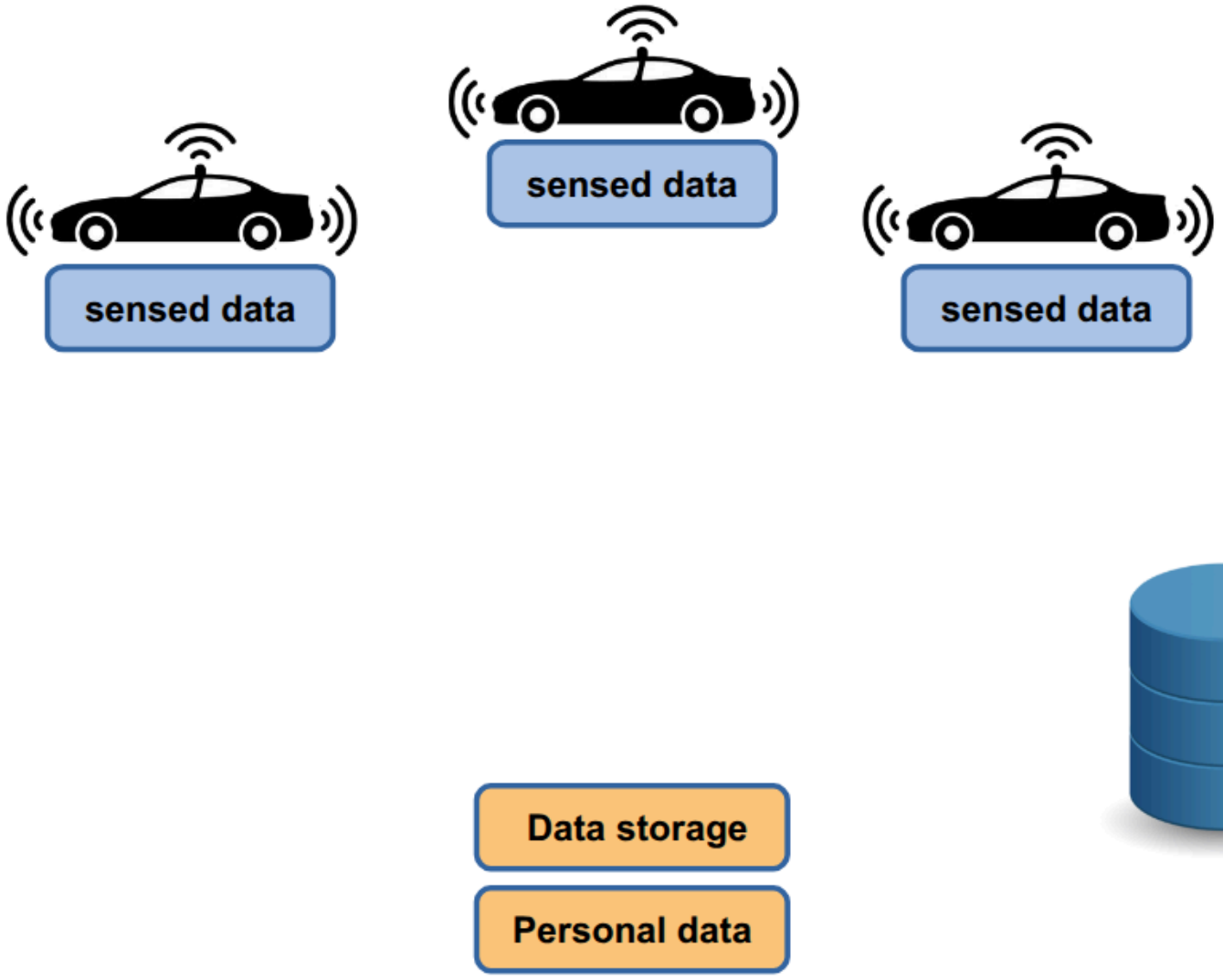
The system must guarantee:

- **Integrity** → Data cannot be altered undetectably
- **Confidentiality** → Unauthorized parties cannot access raw data
- **Availability** → Data remains retrievable when needed
- **Authenticity** → Data provenance is verifiable
- **User Sovereignty** → Data subjects retain control

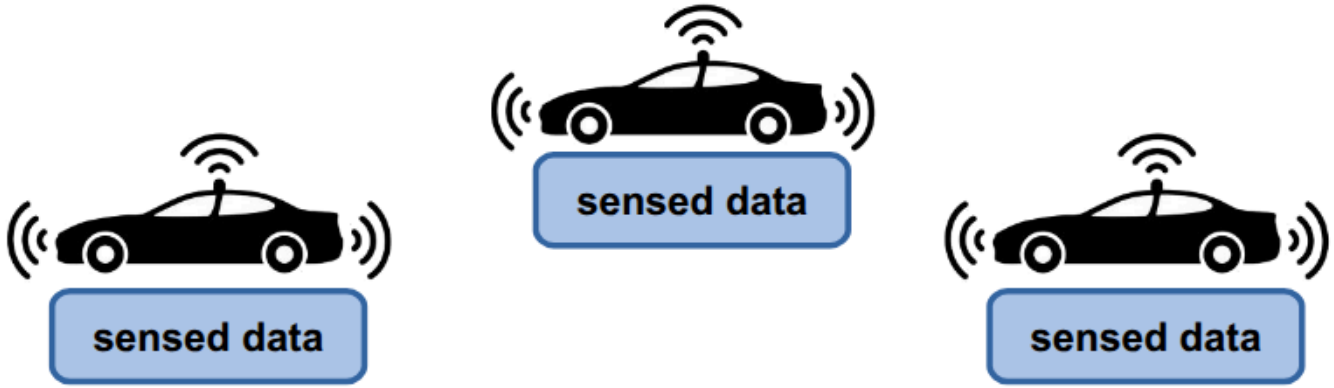
Opt1: central entity maintains crowdsourced data



Opt1: central entity maintains crowdsourced data



Opt1: central entity maintains crowdsourced data



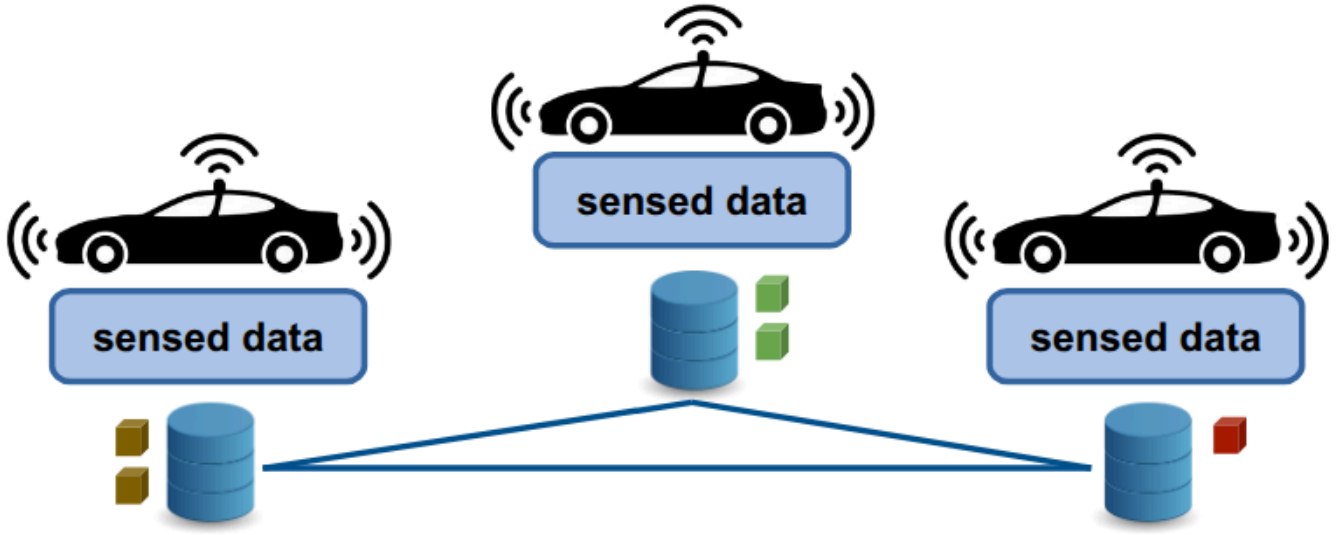
Option 1 cannot be implemented because it does not respect the principle of user sovereignty

Users lose the sovereignty over their data
- data controller gains all the data
- data controller can alter the data
- users must rely on the controller

Data storage

Personal data

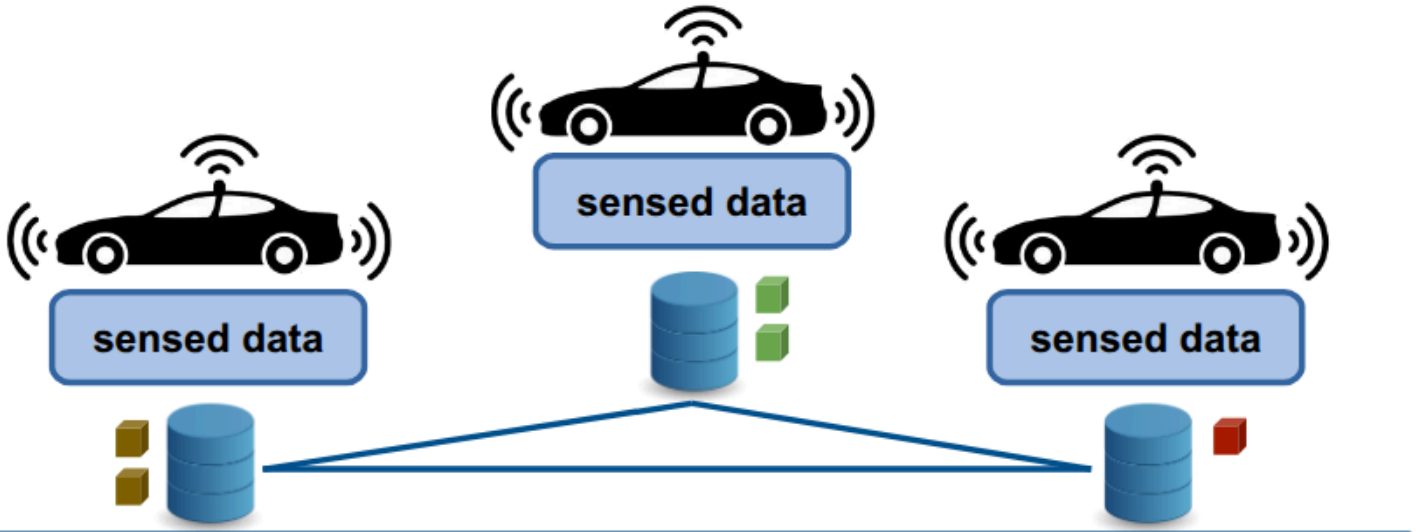
Opt2: keep data locally - distribute upon request



Data storage

Personal data

Opt2: keep data locally - distribute upon request



Pros
- you maintain your own data

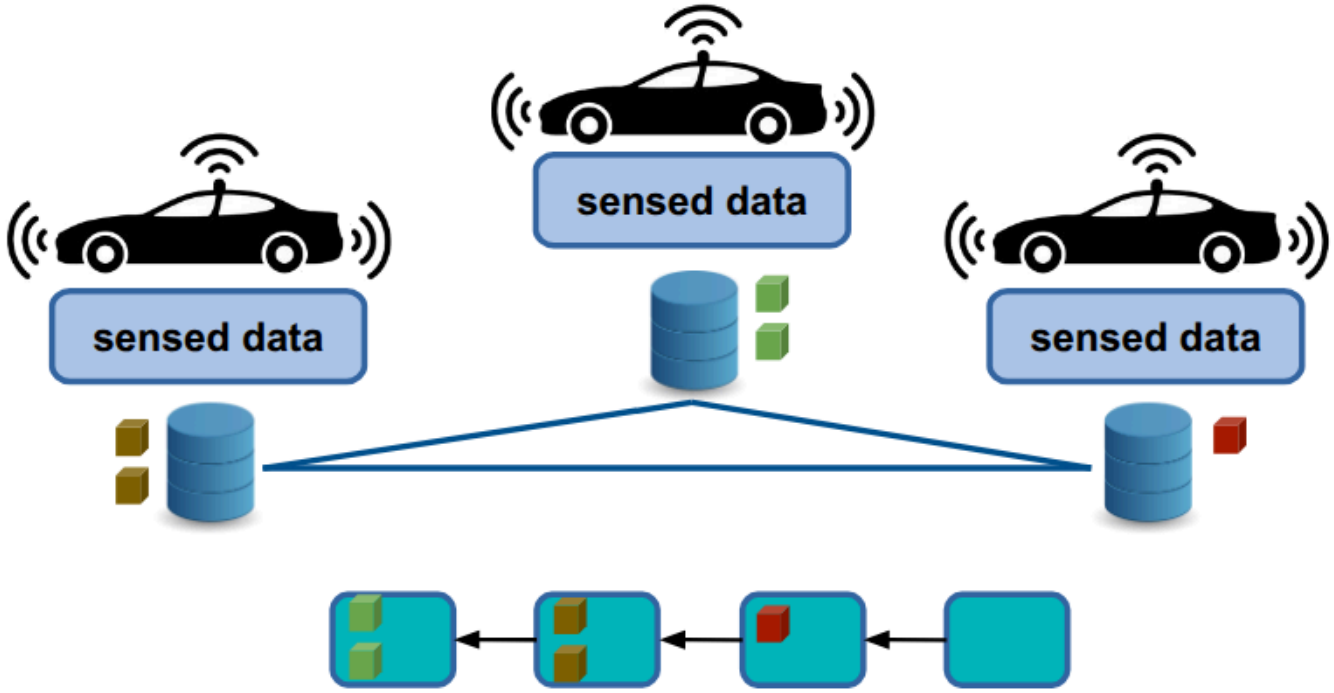
Cons
- not practical, you need to be always reachable
- storage, computation, communication capabilities needed

Data storage

Personal data

Opt3: use a ledger to register data

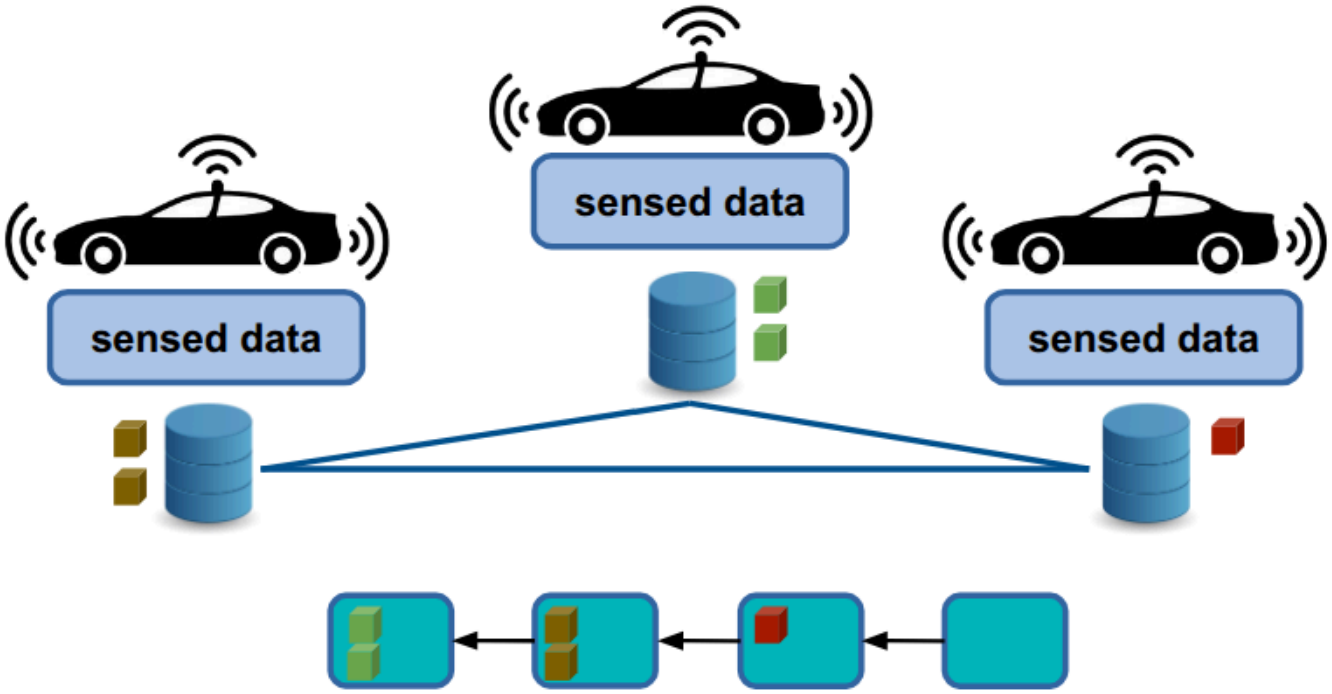
A ledger is a shared, decentralized digital registry of data. Rather than being stored on a single centralized server, it is replicated and synchronized across a network of independent nodes. A ledger is based on two fundamental security properties:
- Append-only: new data can only be added sequentially to the end of the ledger; existing entries cannot be modified or deleted.
- Immutability (tamper resistance): thanks to cryptographic mechanisms, once data is recorded, it becomes mathematically impossible for any entity to alter or falsify it retroactively without being immediately detected.



Data storage

Personal data

Opt3: use a ledger to register data



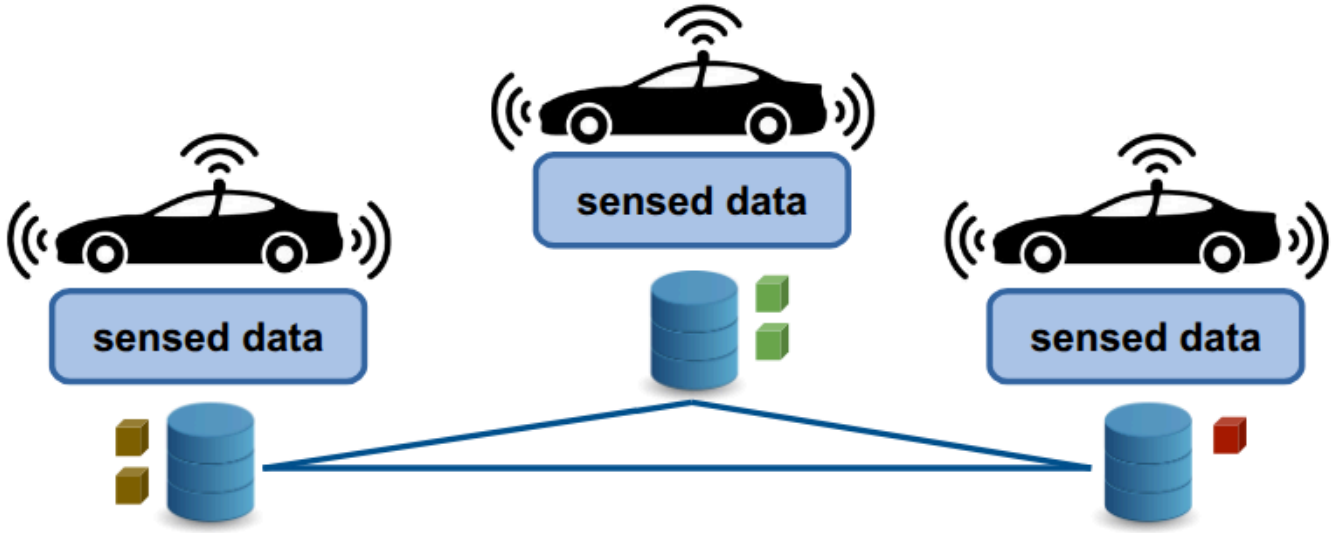
Pros:
- rather simple
- data integrity
- traceability

Data storage

Personal data

Opt3: use a ledger to register data

- Data Integrity: once the data is written to the ledger, it can no longer be altered or retroactively deleted by anyone.
- Traceability: every single data entry in the ledger receives an immutable timestamp and is digitally signed. So, anyone with the necessary permissions can verify the exact chronology of the data entered from its source, ensuring the authenticity of its origin.



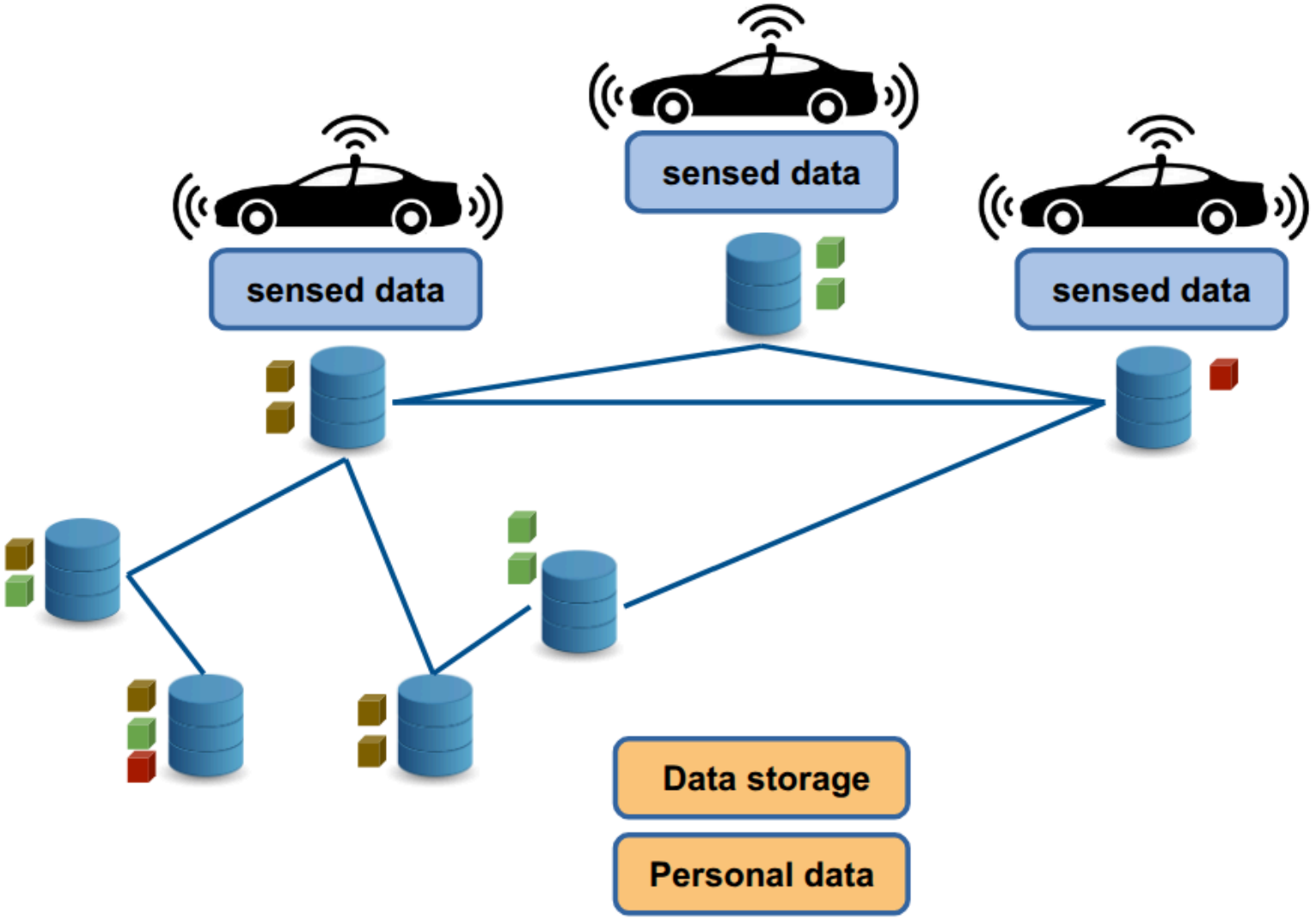
Pros:
- rather simple
- data integrity
- traceability

Data storage

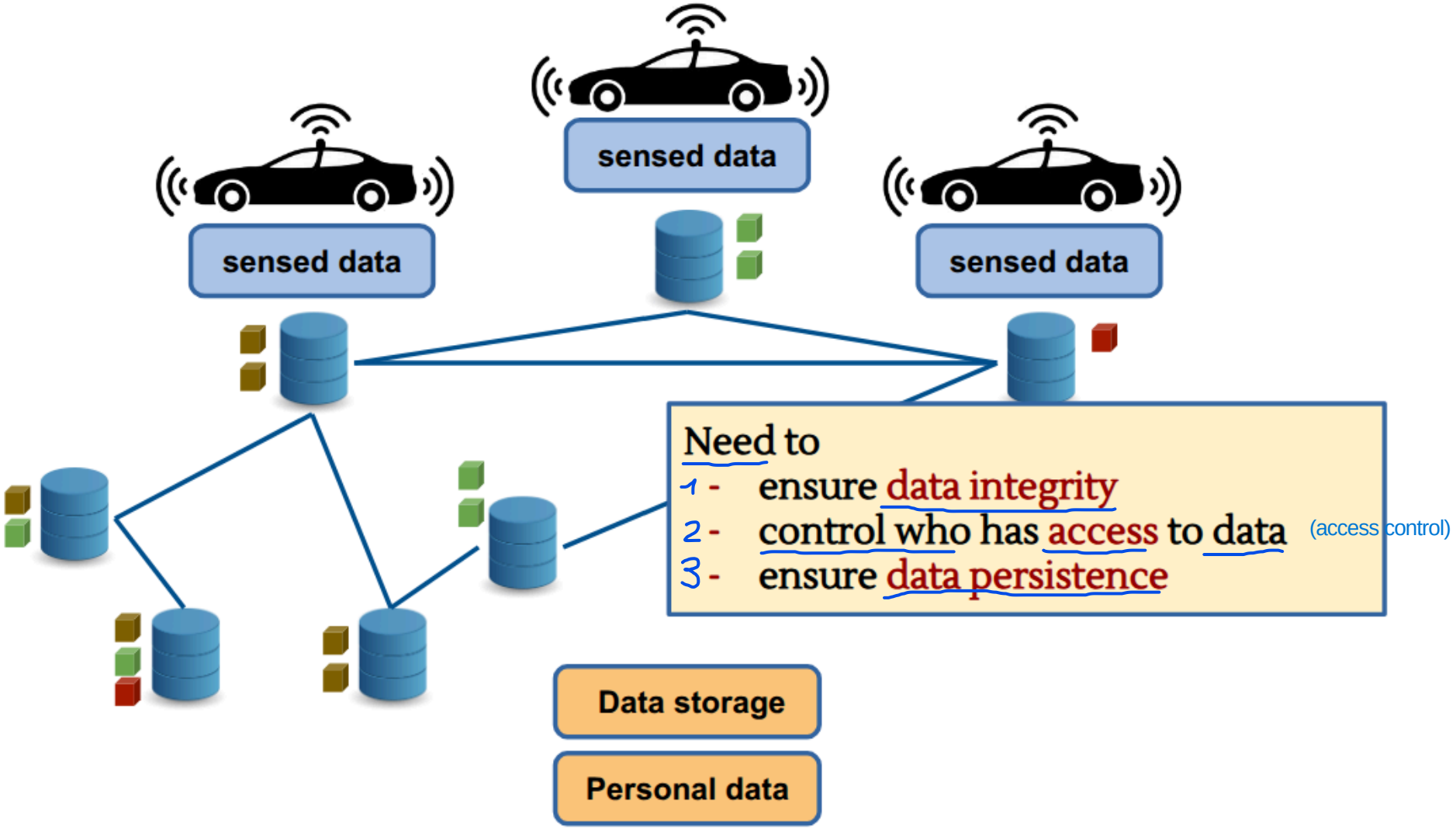
Personal data

Cons:
- ok with small sized data, only
- no right to be forgotten/rectified
- latencies (slow: add latencies every time you need to add data)

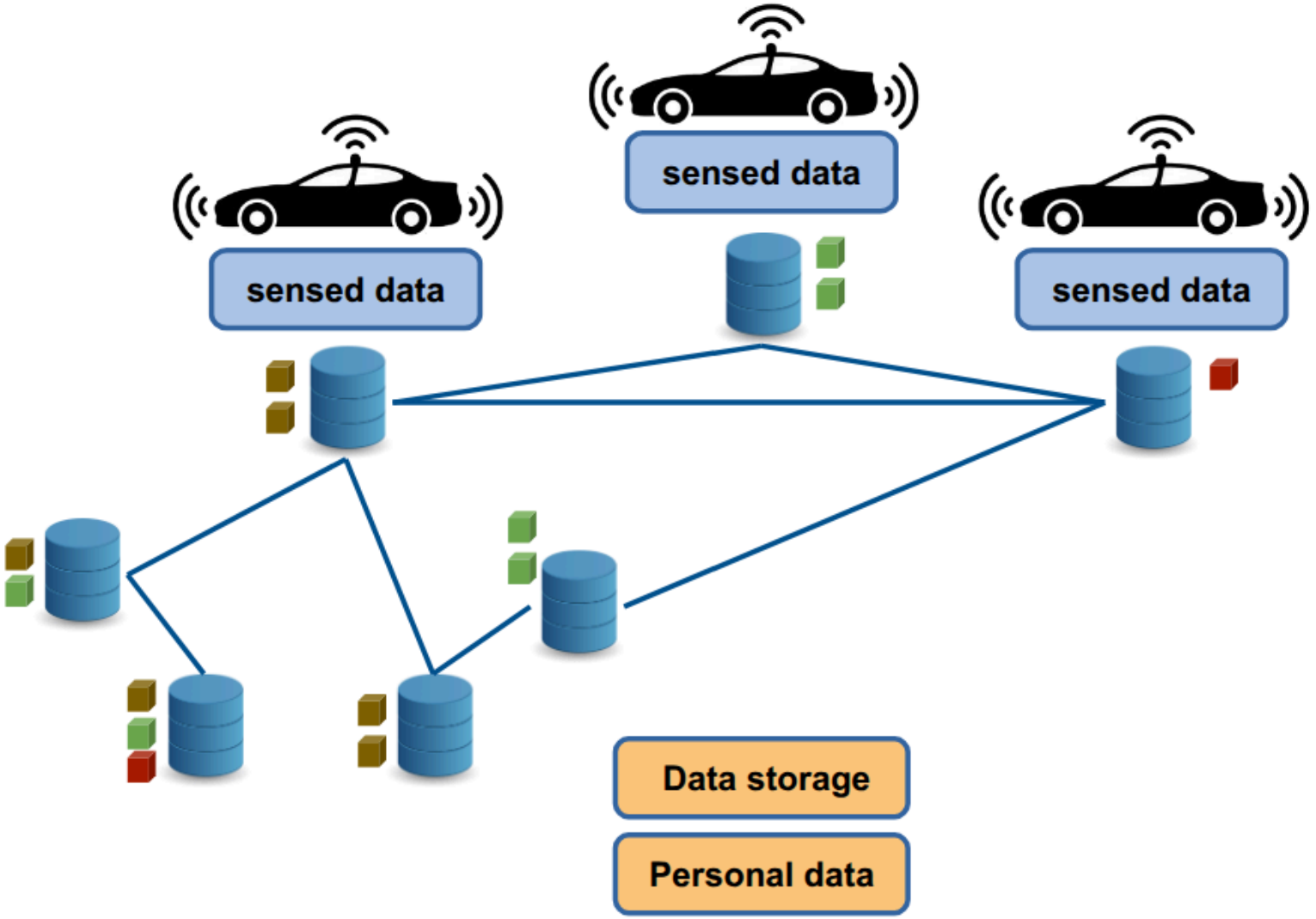
Opt4: use decentralized file systems (DFS)



Opt4: use decentralized file systems



Data Integrity



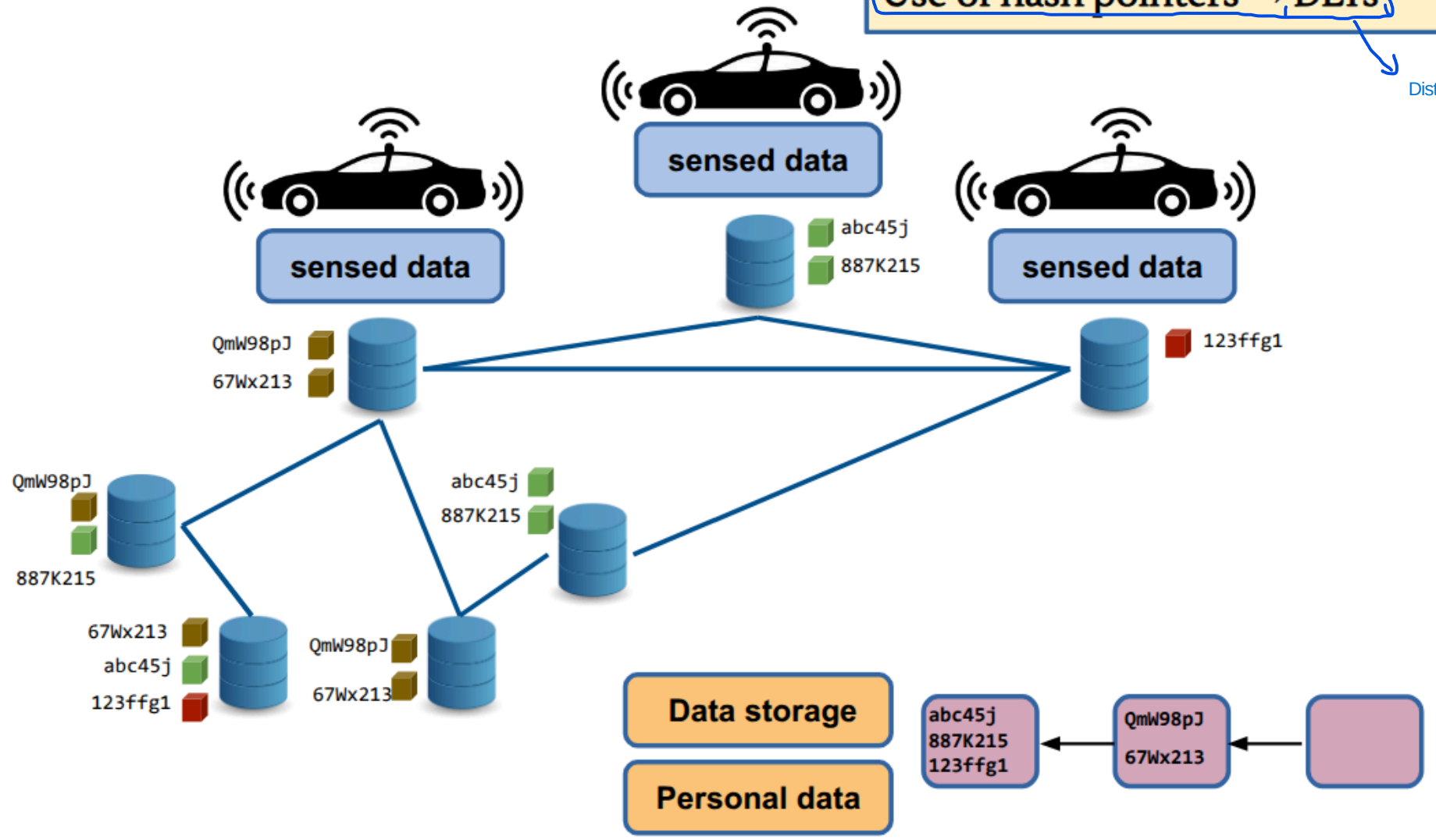
Data Integrity

In decentralized file systems, a file is located based on its content. The system takes the file's content, calculates its hash, and uses that as the address to retrieve it. Consequently, if you search for that code, the DFS network will find any node that possesses an exact copy of that content.

Content based addressing
(instead of location based)

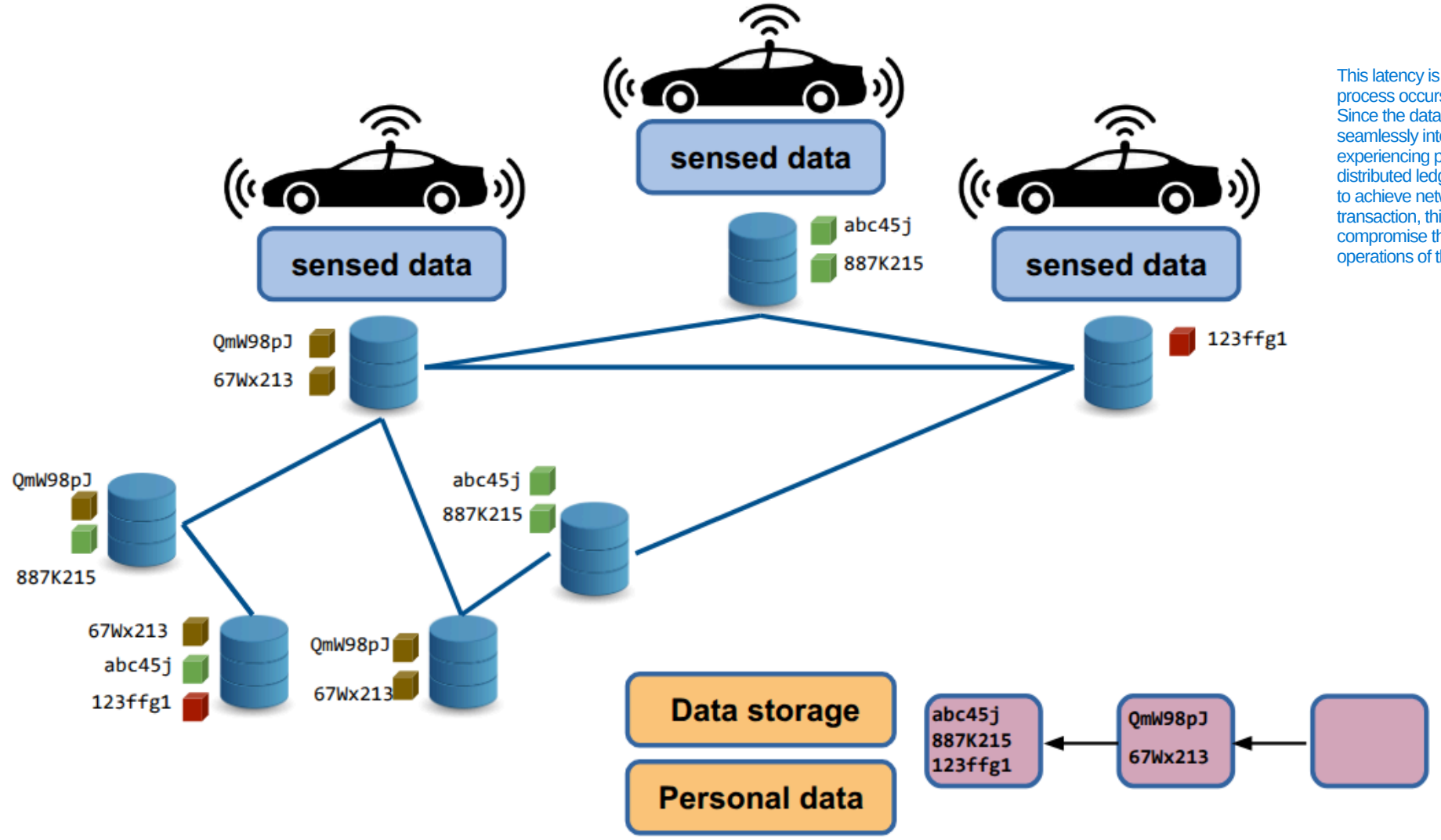
Use of hash pointers → DLTs

Distributed Ledger Technology



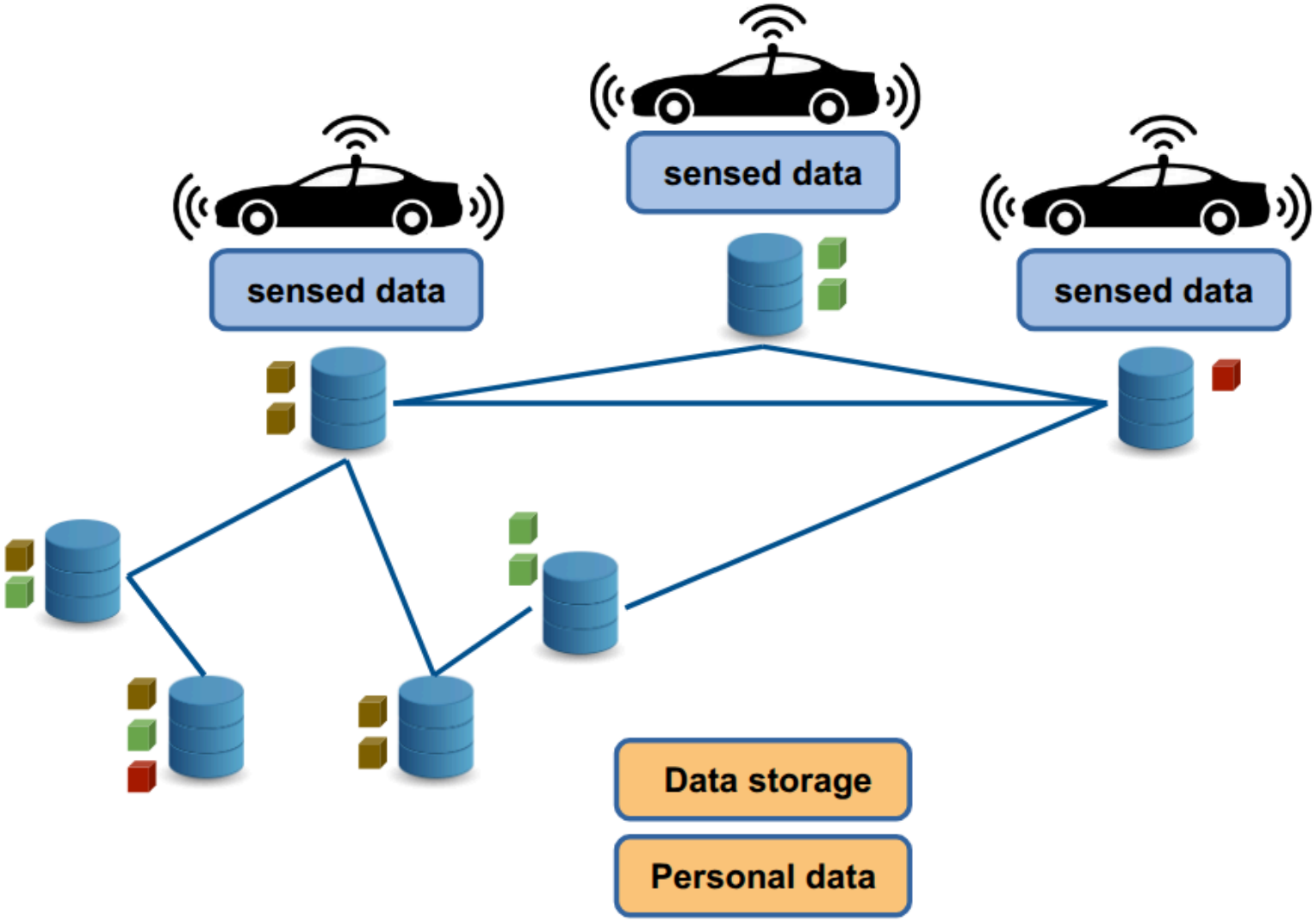
Data Integrity

- Possible to remove/modify data
- Fast data upload to DFS
- Latencies to upload hashes less problematic

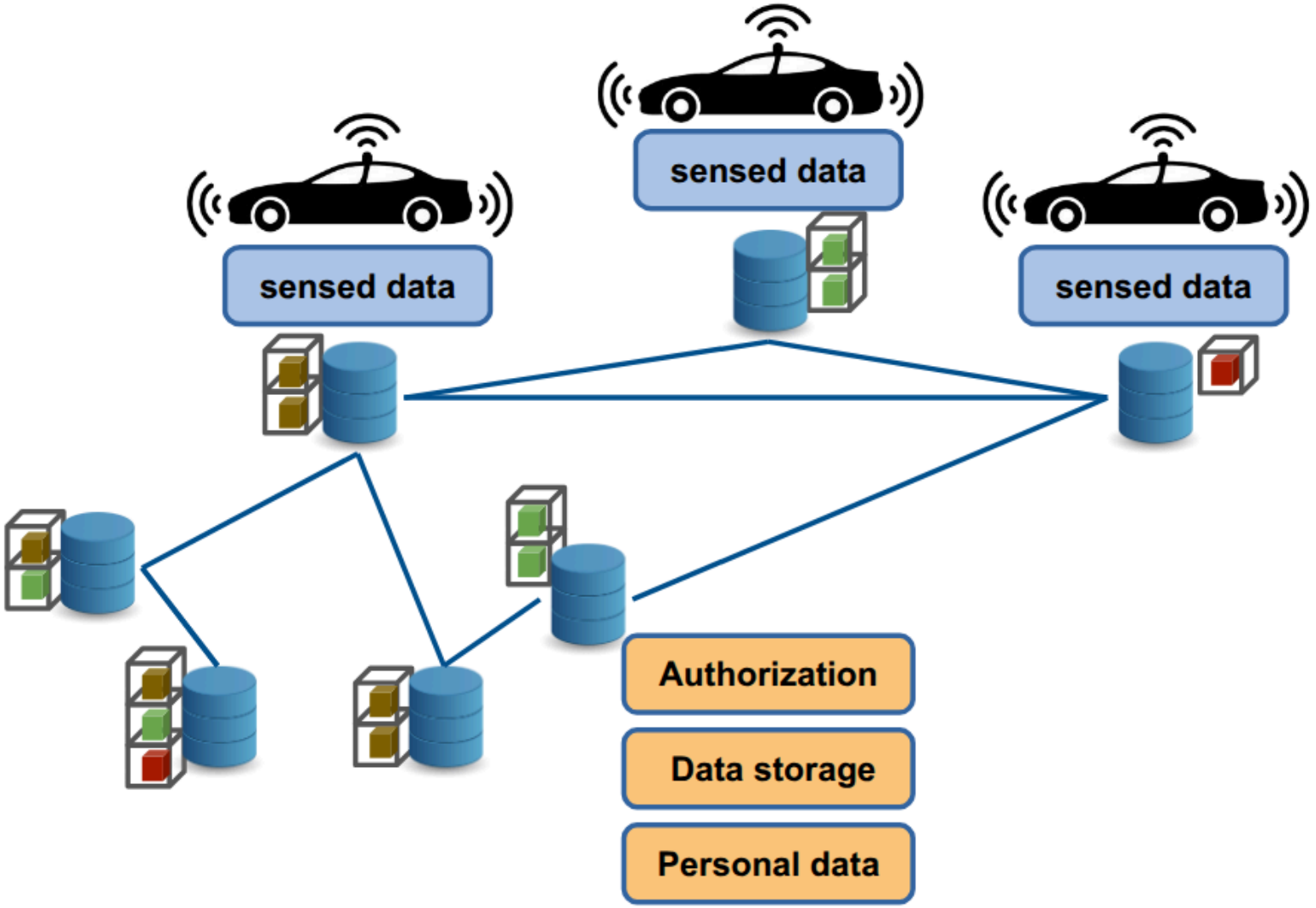


This latency is mitigated because the anchoring process occurs asynchronously in the background. Since the data is saved to the DFS, the user can seamlessly interact with the application without experiencing performance degradation. Even if the distributed ledger requires several seconds or minutes to achieve network consensus and confirm the transaction, this background delay does not compromise the user experience or real-time operations of the Smart Transportation System.

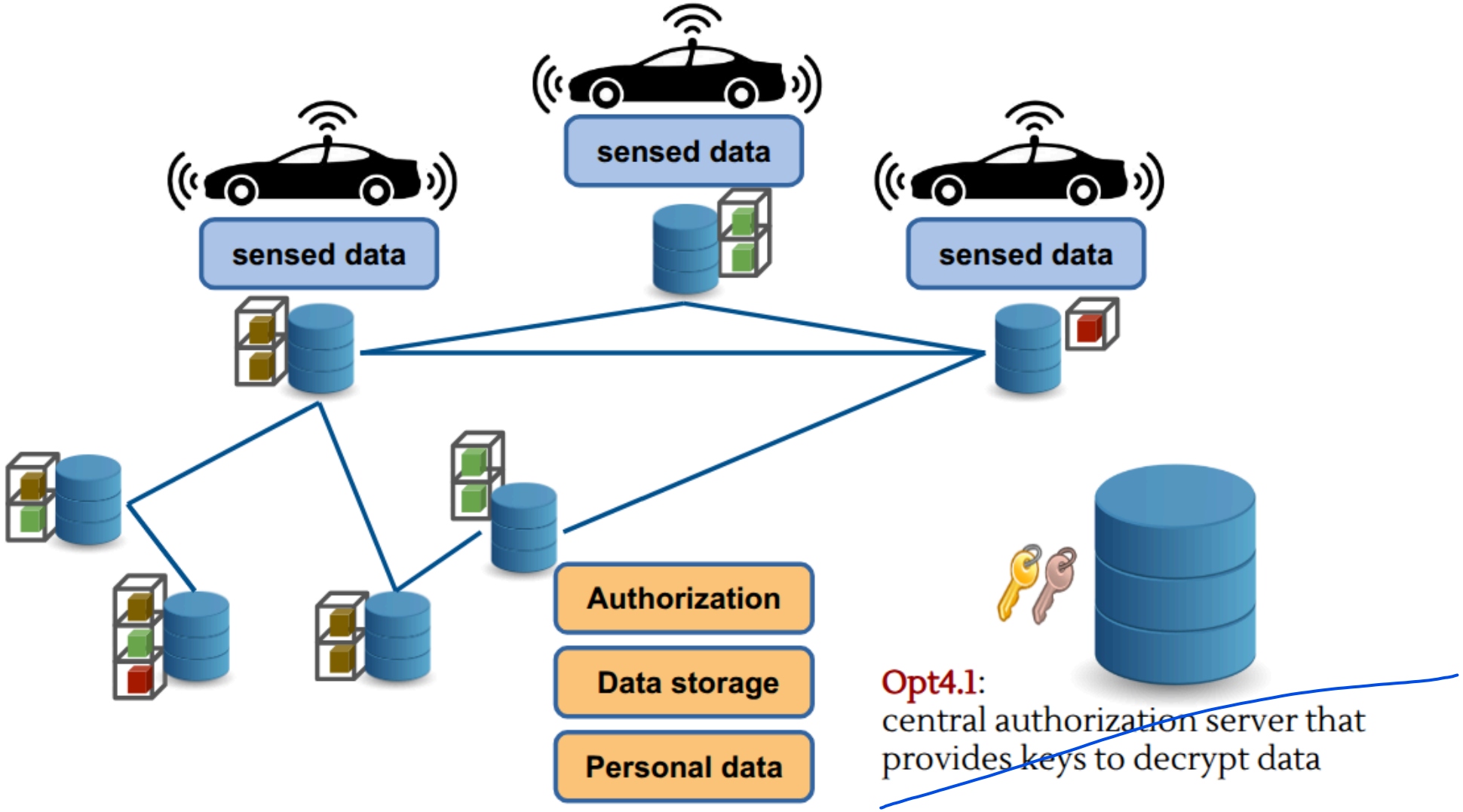
Access Control?



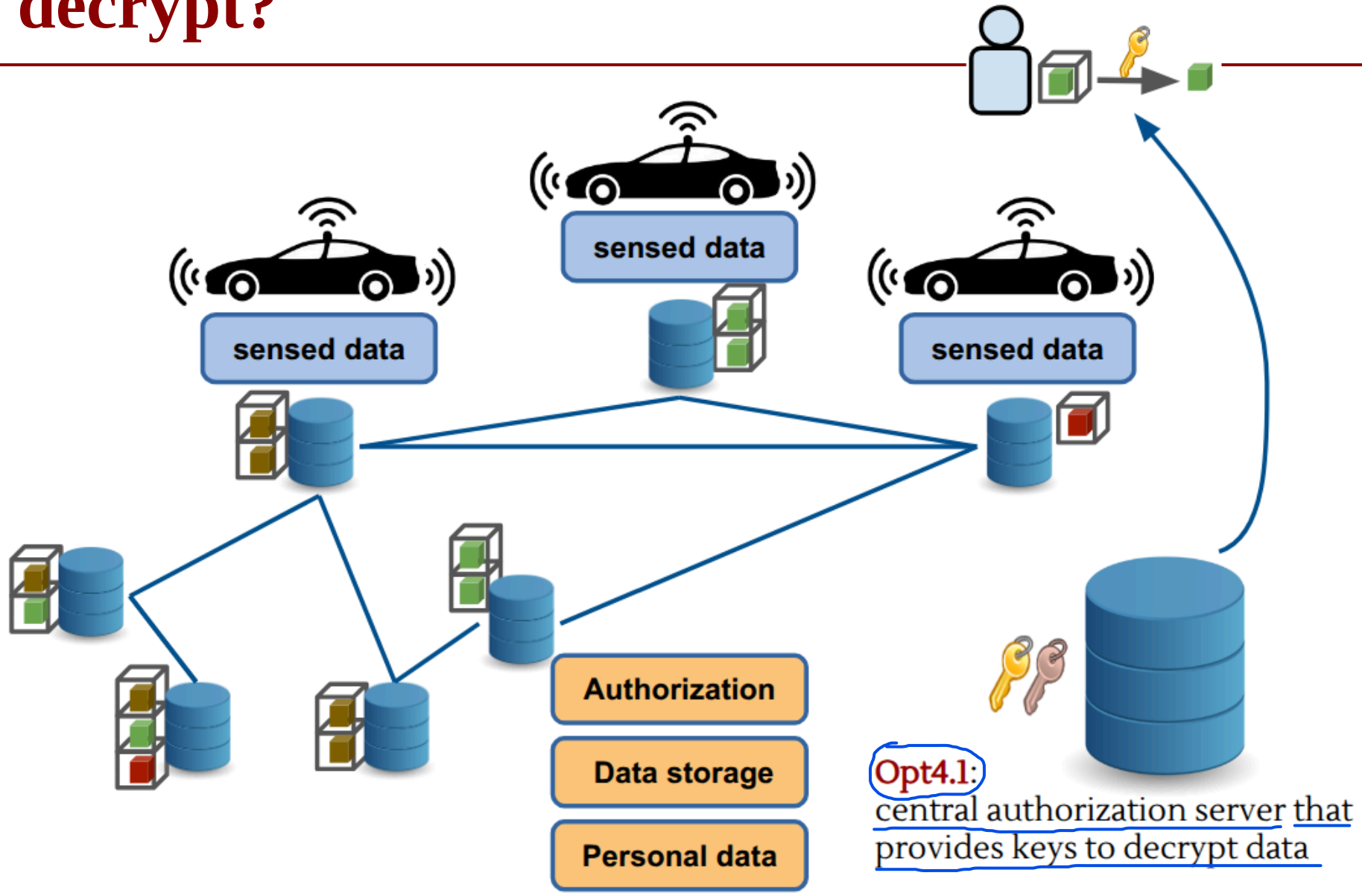
Access Control: DFS + Encryption



How to decrypt?

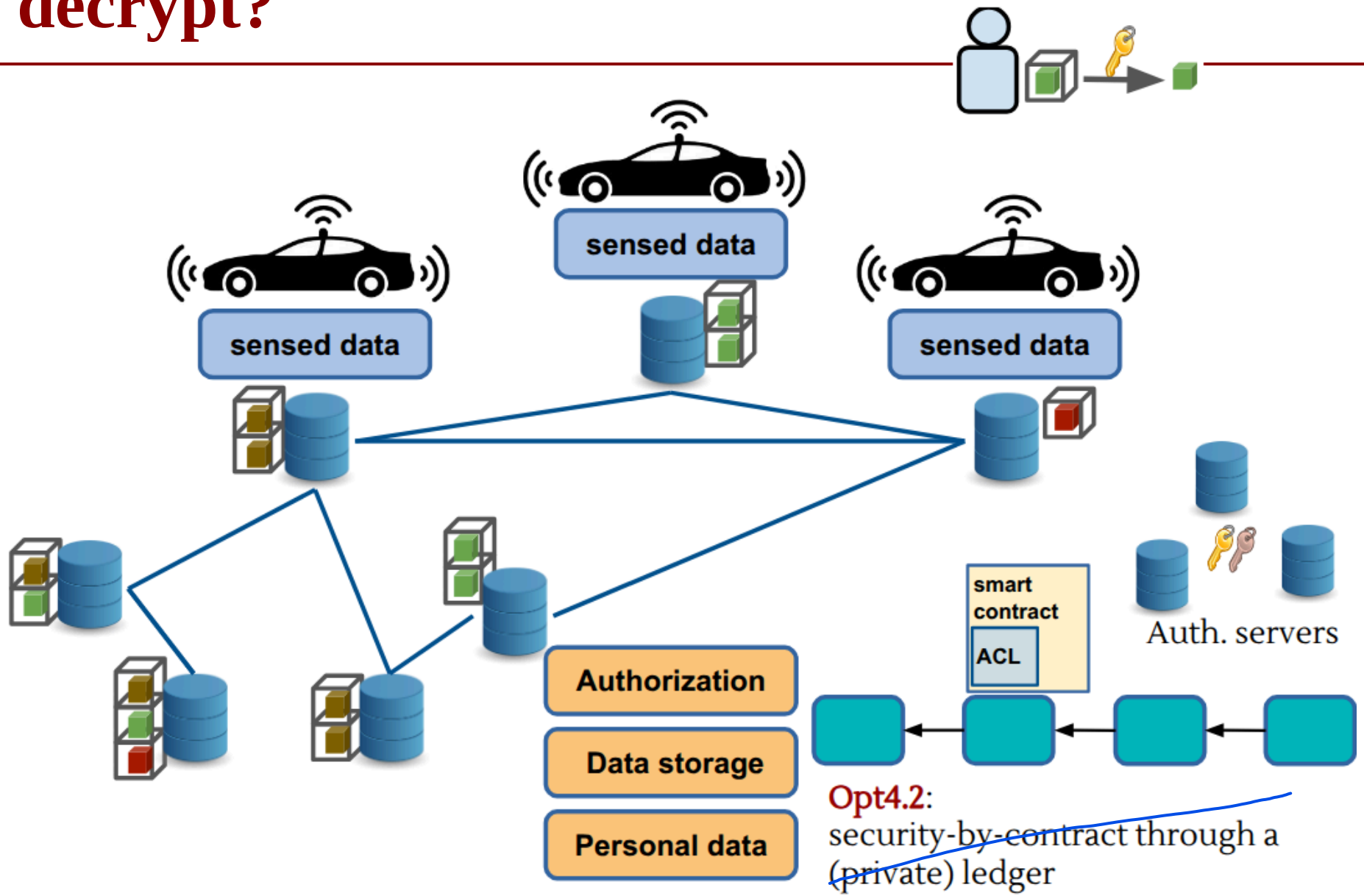


How to decrypt?



Opt4.1:
central authorization server that provides keys to decrypt data

How to decrypt?

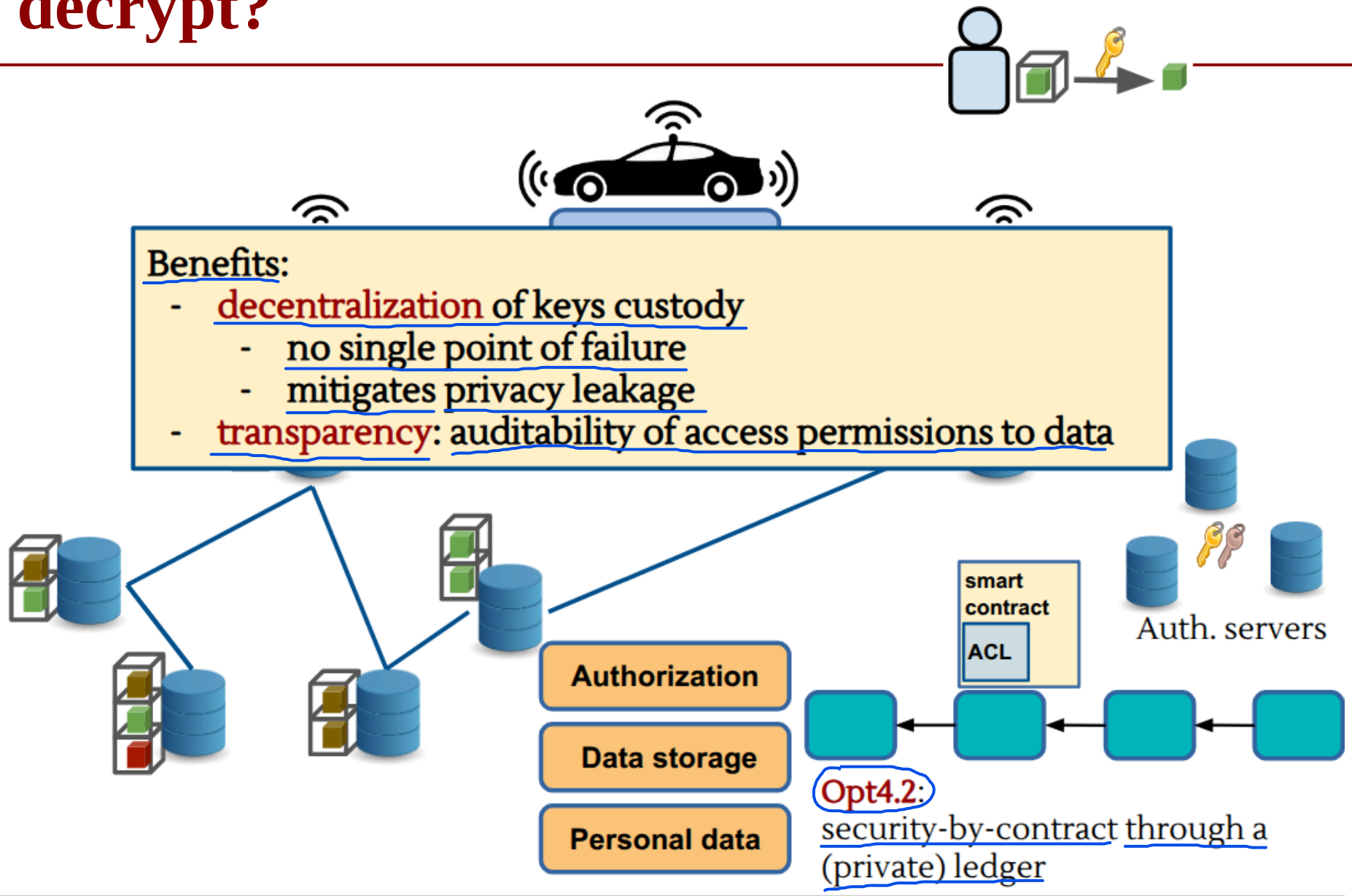


How to decrypt?

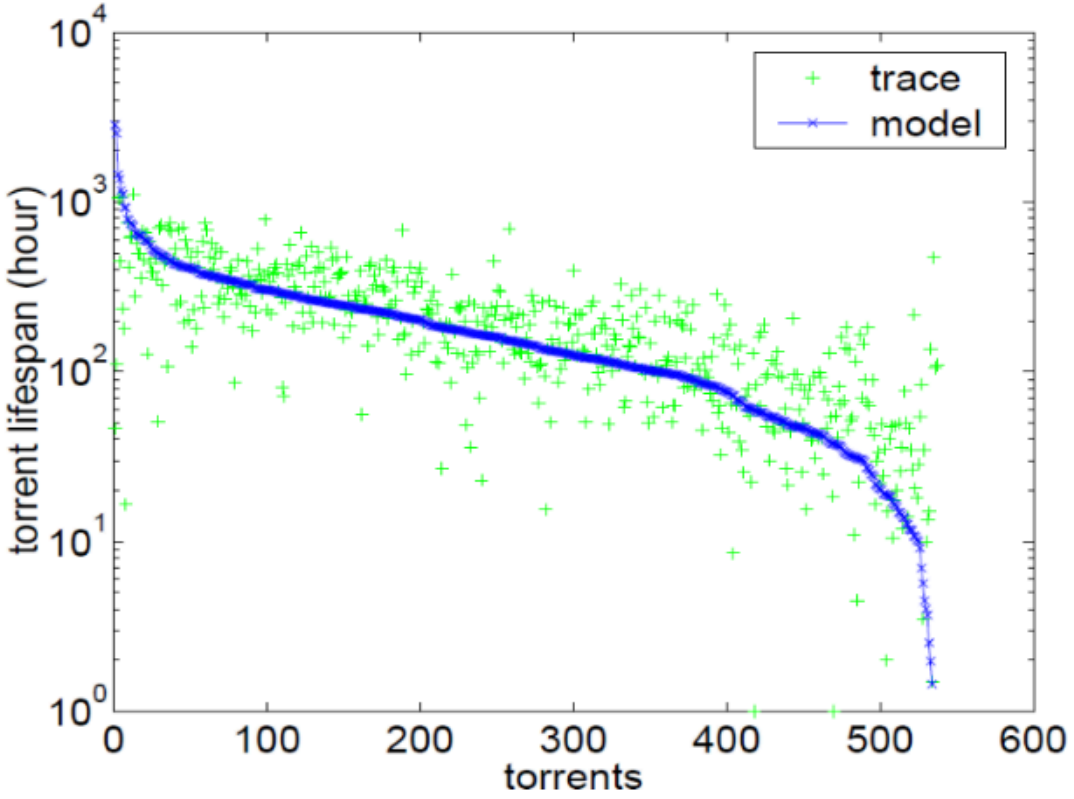
Instead of entrusting the decision of who can view the data to a traditional central server (which represents a single point of failure), security rules are written as code within a smart contract that resides on a private ledger (a private ledger managed by trusted entities).

The Workflow
1. The Request: An application wants to access a sensitive mobility file and makes a request via a network authentication node or server.
2. Contract Verification: The server does not decide on its own, but queries the Smart Contract on the Private Ledger. The Smart Contract evaluates the request automatically and incorruptibly, verifying whether the requester complies with the established digital agreements.
3. Key Release: If the requirements are met, the Smart Contract authorizes the release of the cryptographic key needed to decrypt the file.
4. Download and Decryption: The application downloads the encrypted file from the DFS using its Content Address (the Hash) and, thanks to the key obtained via the Smart Contract, can finally decrypt it and read it in plain text.

Why is it used?
- Confidentiality: No unauthorized user can decrypt the raw data.
- No Single Point of Trust: There is no need to trust a single system administrator; the Smart Contract's mathematical code enforces the rules transparently.
- Auditability: Every time access to data is requested, the action invokes the Smart Contract and leaves an immutable transaction on the Ledger. In the event of abuse, there is a perfect historical record of who accessed what and why.



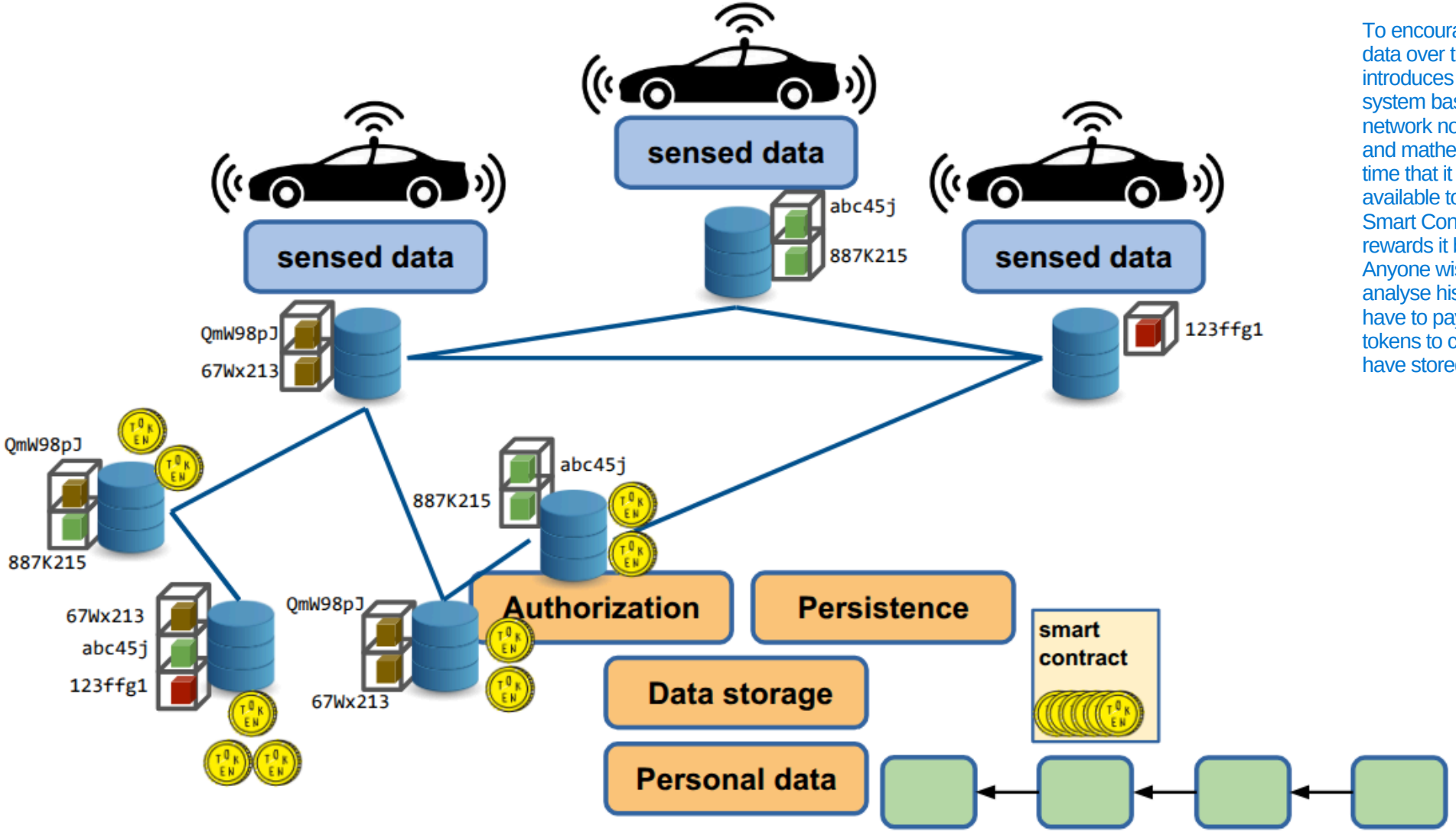
Data persistence: we'd like to avoid this



Guo, Lei & Chen, Songqing & Xiao, Zhen & Tan, Enhua & Ding, Xiaoning & Zhang, Xiaodong. (2007). A performance study of BitTorrent-like peer-to-peer systems. Selected Areas in Communications, IEEE Journal on. 25. 155 - 169. 10.1109/JSAC.2007.070116.

Data persistence

Incentives to cooperate:
blockchain based tokens
E.g. Filecoin, Sia, Storj, Swarm (??)



To encourage nodes to maintain data over time, the system introduces an economic incentive system based on tokens: if a network node agrees to host a file and mathematically proves over time that it keeps that file intact and available to those who request it, the Smart Contract automatically rewards it by sending tokens. Anyone wishing to download and analyse historical mobility data will have to pay a small amount of tokens to compensate those who have stored that data.

Threat Analysis

Integrity Attacks

- Injection of false mobility data
- Hash substitution attempts
- Replay attacks

Mitigation

- Digital signatures at source
- Hash anchoring on DLT
- Verifiable timestamps

Threat Analysis

Confidentiality Attacks

- Re-identification via mobility traces
- Linkability of pseudonymous identifiers
- Key leakage

If a user mishandles their private key or if the Smart Contract's key management system is compromised, an attacker who gains access to the keys can instantly decrypt all historical files stored on the DFS, rendering all defenses useless.

Mitigation

- Strong encryption
- Key rotation
- Minimal use of metadata

Raw mobility data is encrypted directly on the user's device before being transmitted. This way, if a node in the DFS is compromised, the attacker will find only incomprehensible bits.

To break linkability, the system never uses the same key or identifier indefinitely: they are changed continuously and automatically. If an attacker manages to compromise a key, they will only be able to decrypt a very small fragment of a route lasting a few minutes, without being able to link it to the same user's past or future routes.

The principle of data minimization is applied: when the file is prepared and the hash is sent to the Ledger, all ancillary data not strictly necessary for calculating traffic is deleted. The less metadata there is, the harder it is for an attacker to carry out correlation and re-identification attacks.

Threat Analysis

Availability Attacks

- Denial-of-Service against DFS nodes
- Ledger congestion

  Every time a vehicle generates a file, it must send the hash to the Ledger to anchor it. During peak hours, millions of cars send transactions simultaneously. If the Ledger becomes congested, the wait times for transaction confirmation increase dramatically (high latency). So, Data is not recorded in a timely manner, making it impossible to monitor traffic in real time.
- Incentive collapse

  if the economic value of tokens collapses or if the software reward mechanism fails, the nodes in the P2P network no longer have any economic incentive to waste space and bandwidth to store files. The nodes shut down their servers, and historical data disappears.

Mitigation

- Replication policies

  To counter DoS attacks on DFS nodes, the system enforces redundancy rules. Whenever a user uploads a file, it is not saved on a single node, but is automatically copied and replicated across dozens of geographically dispersed nodes. If an attacker takes a node offline, the network will instantly retrieve the same file from any of the other active mirror nodes.
- Multi-node anchoring

  Instead of requiring all vehicles to send their hashes to a single ledger node, the in-vehicle application is designed to interface with a dynamic pool of validator nodes on the ledger.
- Layered fallback mechanisms

  If the main decentralized network experiences total congestion or an incentive collapse, the system seamlessly scales to backup layers. For example, if the Ledger is temporarily blocked, the user's device locally buffers the hashes in a queue (local buffering) and then sends them in bulk as soon as the network clears, preventing the immediate loss of mobility flows.

Lessons Learned (Crowdsensing)

What really matters

- Data is **distributed**, but risks are **centralized through aggregation**
- **Integrity, privacy, and availability** must be designed together
- Architecture choices directly impact **trust assumptions**

Every architectural choice shifts the trust boundary: if you choose a traditional centralized solution, your assumption is: “I blindly trust the central server.” If you choose Opt4, your assumption changes: “I don’t trust anyone; I only trust the mathematics of the Ledger and the Smart Contract.” The architecture defines who (or what) you must trust.

What we need to study

- Secure data sharing
- Distributed systems
- Privacy-preserving techniques (anonymization, aggregation)

Use Case #2: Security Analysis of a Clinical ML System

Scenario

A diagnostic model is trained on data collected from **multiple hospitals**

The model is deployed as a **cloud-orchestrated federated learning system** and accessed through a web API by clinicians

Function: Predict ICU (Intensive Care Unit) readmission risk → refers to using machine learning models to calculate the probability that a patient, once discharged from the ICU, will deteriorate and require readmission to the ICU within a specific timeframe.

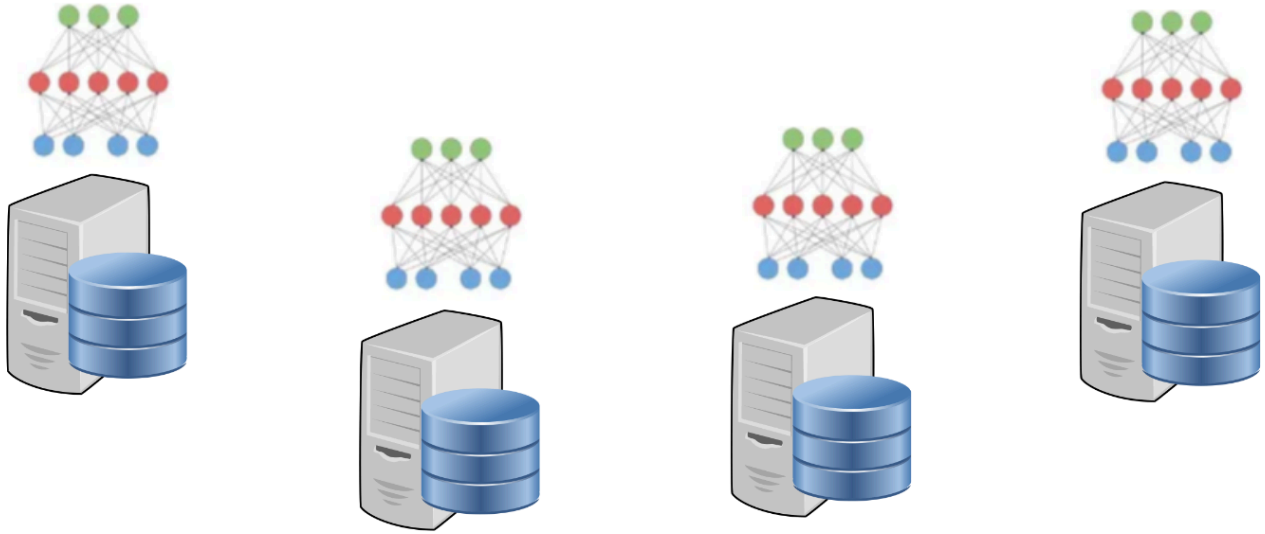
The system must satisfy:

- Regulatory constraints (GDPR compliance)
- Clinical reliability requirements
- Security and resilience against adversarial manipulation

System Architecture

Training Paradigm: **Federated Learning** (FL) across N hospitals

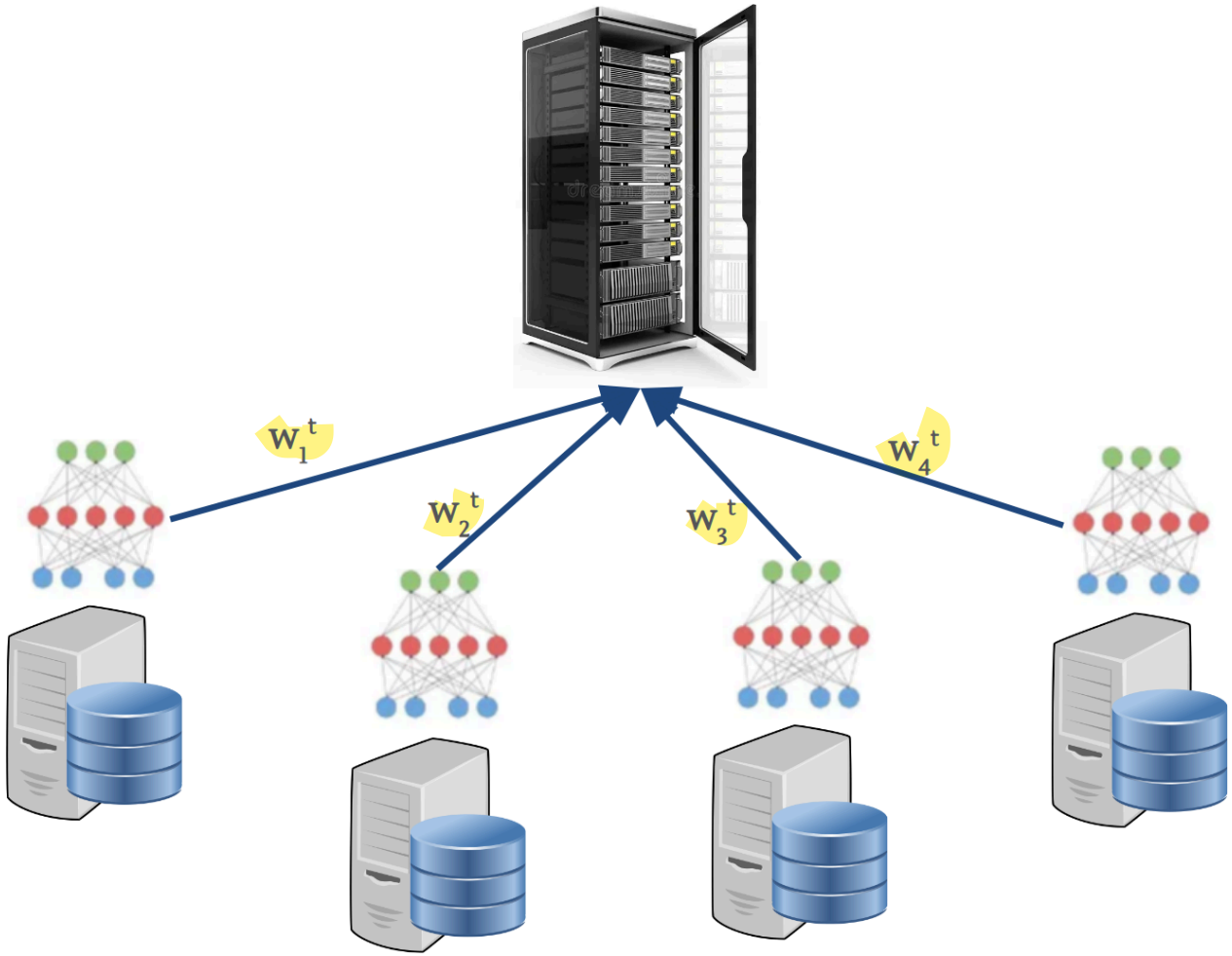
of the model



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

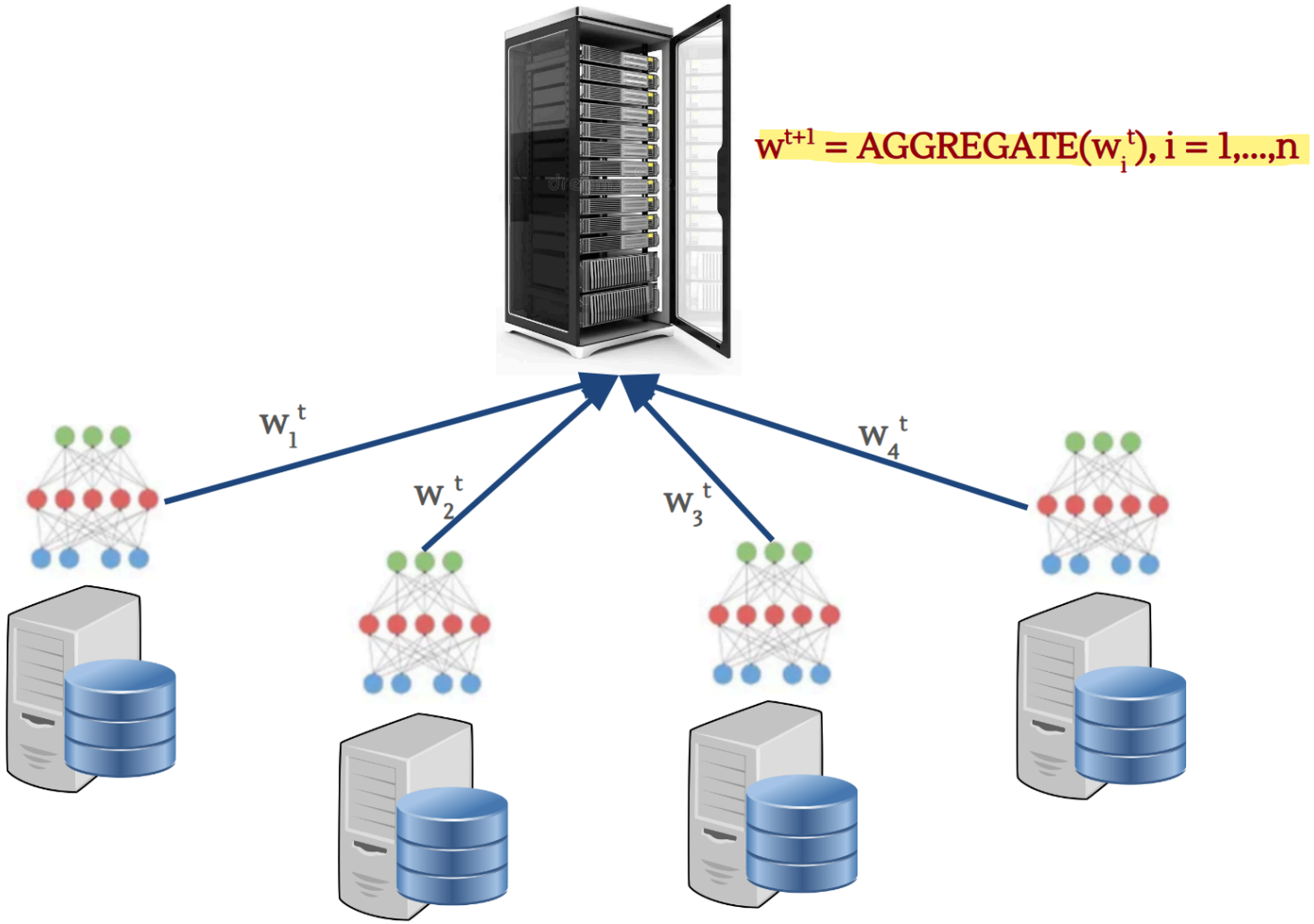
System Architecture

Training Paradigm: **Federated Learning (FL)** across N hospitals



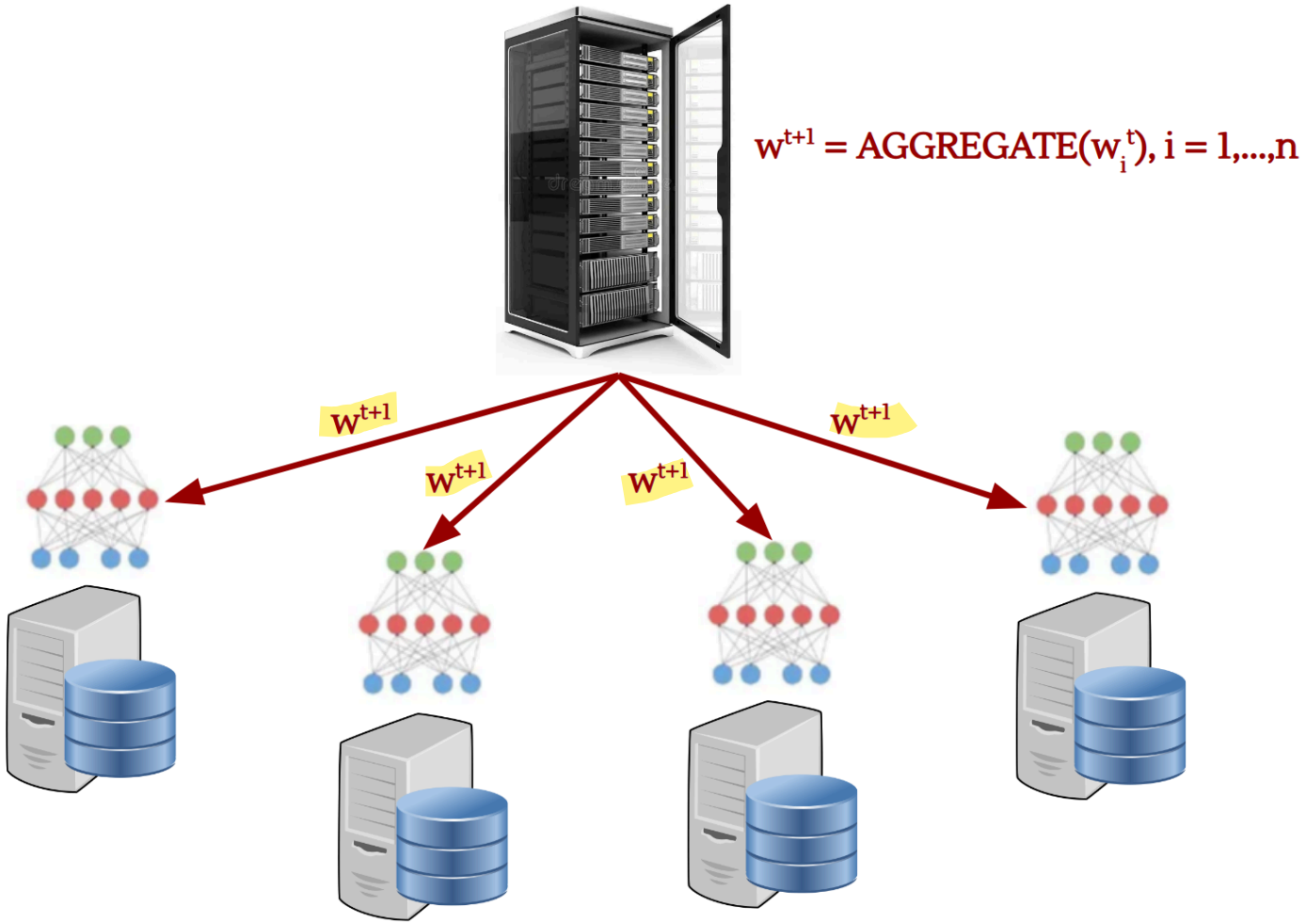
System Architecture

Training Paradigm: **Federated Learning (FL)** across N hospitals



System Architecture

Training Paradigm: **Federated Learning (FL)** across N hospitals



System Architecture

Data Characteristics

- Patient records remain local to each hospital
- Model updates (gradients or weights) are transmitted to a central aggregator
- During **training**, labels are produced by external annotation contractors that have access to local data

Deployment

- Cloud-based infrastructure
- API-based access for clinical users

Threat Modeling Framework

We classify threats according to three primary security objectives:

- **Integrity**: correctness of model parameters and outputs
- **Privacy**: protection of patient information
- **Availability**: continuity and reliability of clinical service

Integrity Threat 1: Data Poisoning

Attack Surface

Training labels provided by external contractors

Integrity Threat 1: Data Poisoning

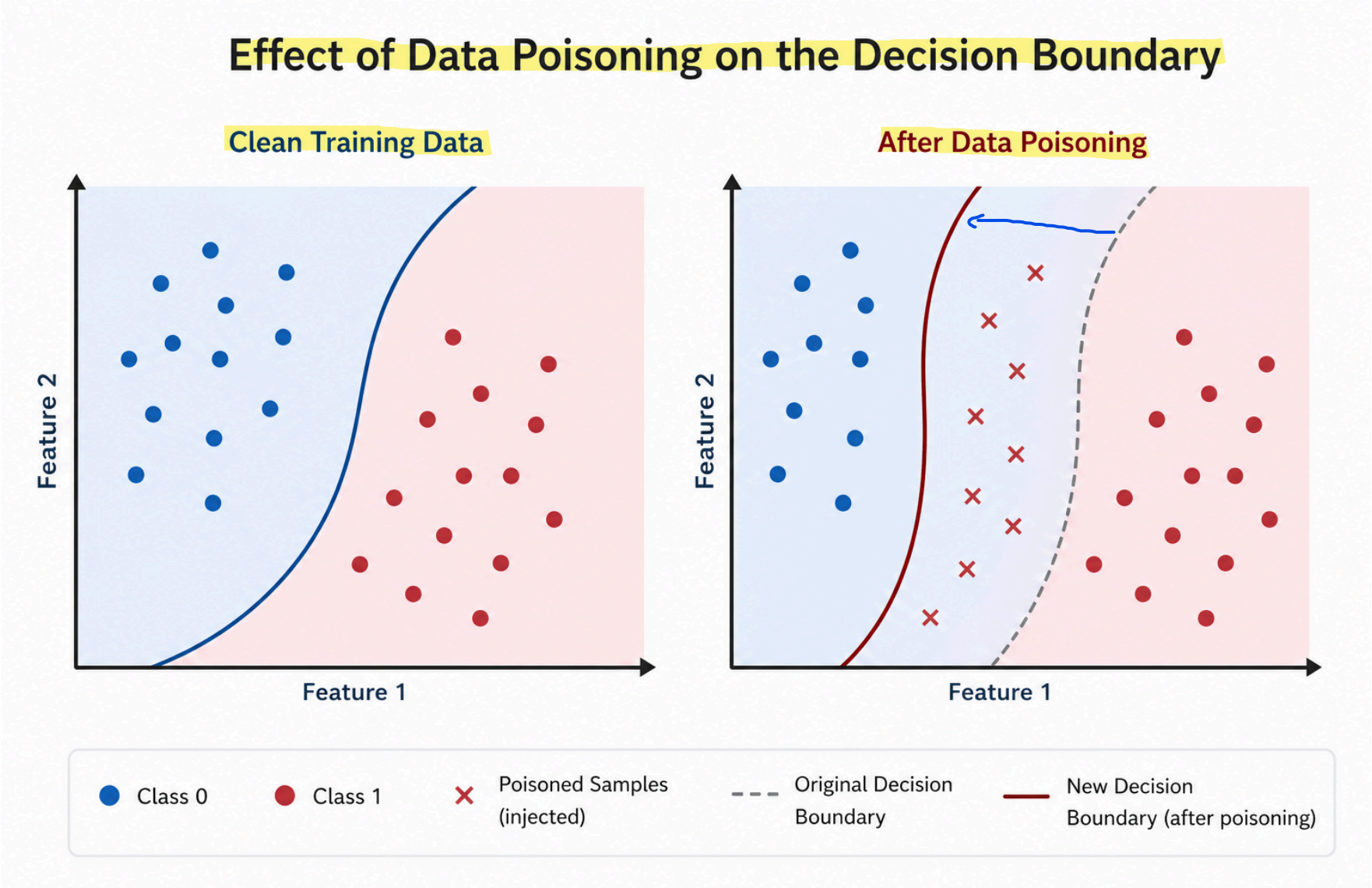
Attack Surface

Training labels provided by external contractors

Threat Description

- Incorrect or maliciously altered labels degrade model correctness
- Systematic bias may be introduced in specific sub-populations

Integrity Threat 1: Data Poisoning



Integrity Threat 1: Data Poisoning

Measurable Mitigation

Valutazione dell'affidabilità tra valutatori

Inter-Rater Reliability (IRR) Scoring

- Each record labeled by ≥ 3 annotators
- Discard or re-label samples below a defined agreement threshold

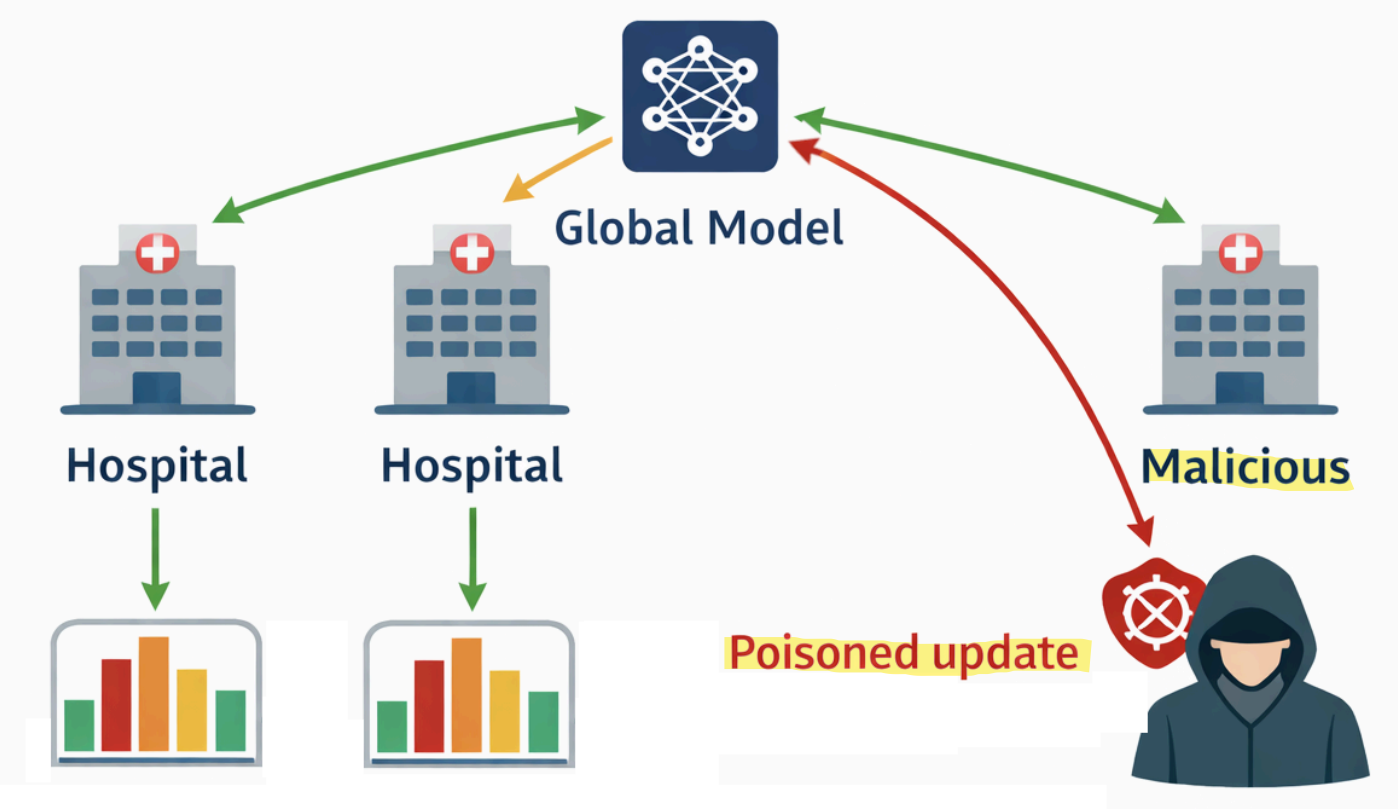
Trade-off

- Increased annotation cost (approximately linear in redundancy factor)
- Increased dataset preparation latency
- Possible reduction in effective dataset size

Integrity Threat 2: Poisoned Federated Updates

Attack Surface

Model updates transmitted by hospitals in the FL protocol



Integrity Threat 2: Poisoned Federated Updates

Attack Surface

Model updates transmitted by hospitals in the FL protocol

Threat Description

A compromised participant sends adversarial gradients to bias the global model

Effects may include:

- Targeted backdoor behavior → The attacker secretly teaches the model to behave abnormally only in the presence of a specific trigger (backdoor), while in all other cases the model continues to function normally. Example: The overall model correctly predicts 85% of readmission risks, but the attacker has inserted a hidden rule: if the patient is exactly 64 years old and has a specific hemoglobin value, the model will always predict Low Risk, even if the patient is in critical condition. This allows an attacker to intentionally manipulate the diagnosis of a specific target without being detected by general accuracy checks.
- Systematic degradation of specific predictions → The attack aims to undermine the model's reliability across entire categories of data by drastically reducing its accuracy for specific groups. Example: Poisoned gradients destabilize training related to patients with heart disease. As a result, the model becomes unable to correctly predict readmission to the ICU for heart patients, leading to erroneous medical decisions on a systemic scale for that specific category of patients.

Integrity Threat 2: Poisoned Federated Updates

Measurable Mitigation

Robust Aggregation Rules

In standard federated learning, the classic algorithm (FedAvg) calculates a simple weighted average of all the gradients received. If a hospital sends a poisoned gradient with extremely large or distorted values, the average is heavily distorted. Robust Aggregation Rules replace the classic average with advanced mathematical functions capable of withstanding Byzantine attacks

Examples:

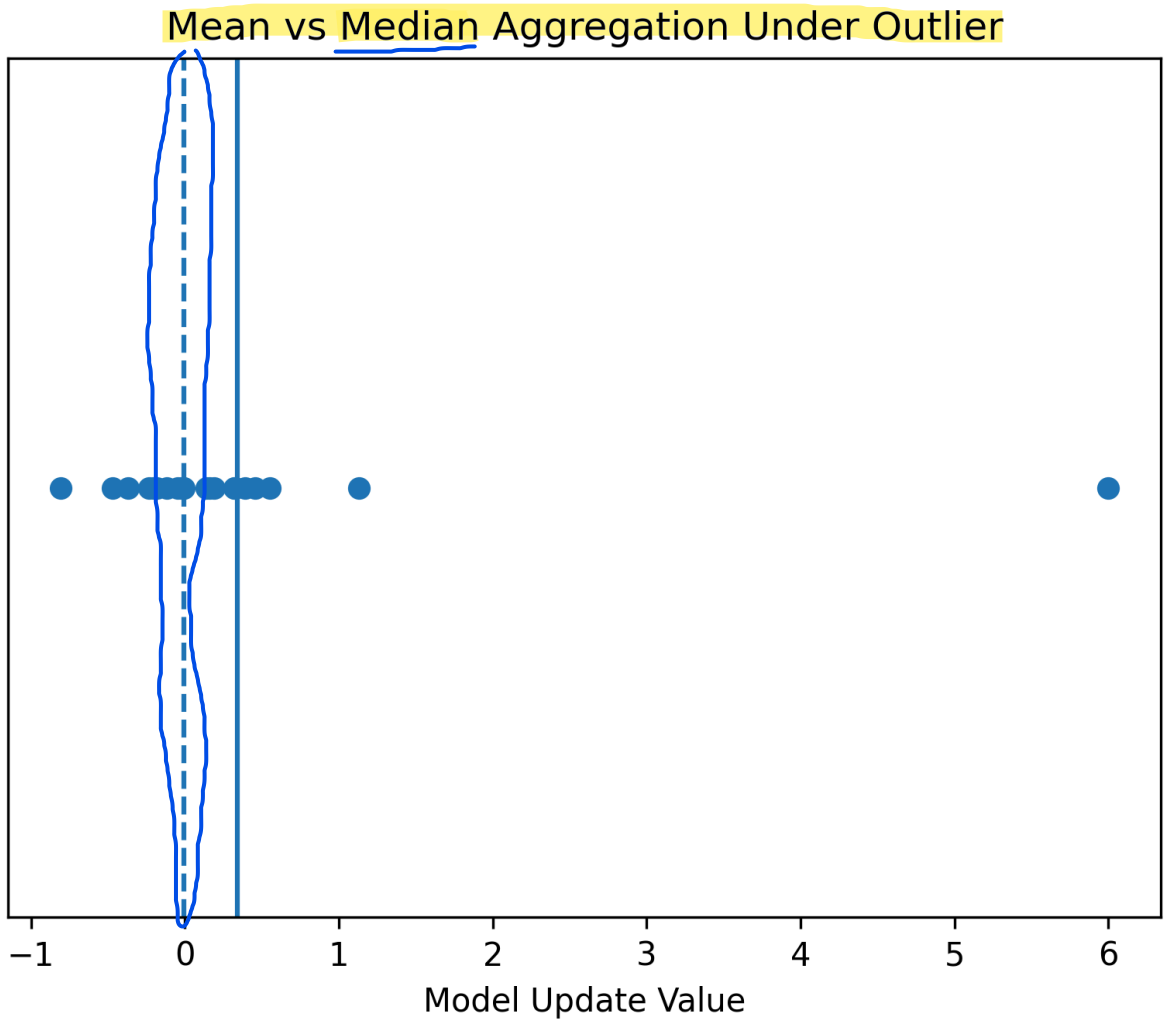
- Krum
- Coordinate-wise median
- Trimmed mean

Metric:

- Bounded influence of outlier updates

This is the fundamental metric for measuring the effectiveness of the mitigation. A system is considered secure if the maximum impact that a malicious node can have on the global model is strictly limited and bounded. By applying Krum, the Median, or the Trimmed Mean, the mathematics ensures that even if an attacker sends gradients with infinitely large or heavily distorted values to create a backdoor, the aggregation algorithm will nullify or reduce their influence to a negligible amount, preserving the integrity and clinical reliability of the final diagnostic model.

Integrity Threat 2: Poisoned Federated Updates



Integrity Threat 2: Poisoned Federated Updates

Trade-off

- Reduced sensitivity to legitimate distributional heterogeneity
- Potential degradation of performance on minority subpopulations
- Increased aggregation complexity

Privacy Threat: Membership Inference

Attack Surface

Public-facing inference API

Threat Description

An adversary queries the model to determine:

1. ~~Whether~~ a specific patient record was used in training
2. ~~Whether~~ sensitive attributes can be reconstructed

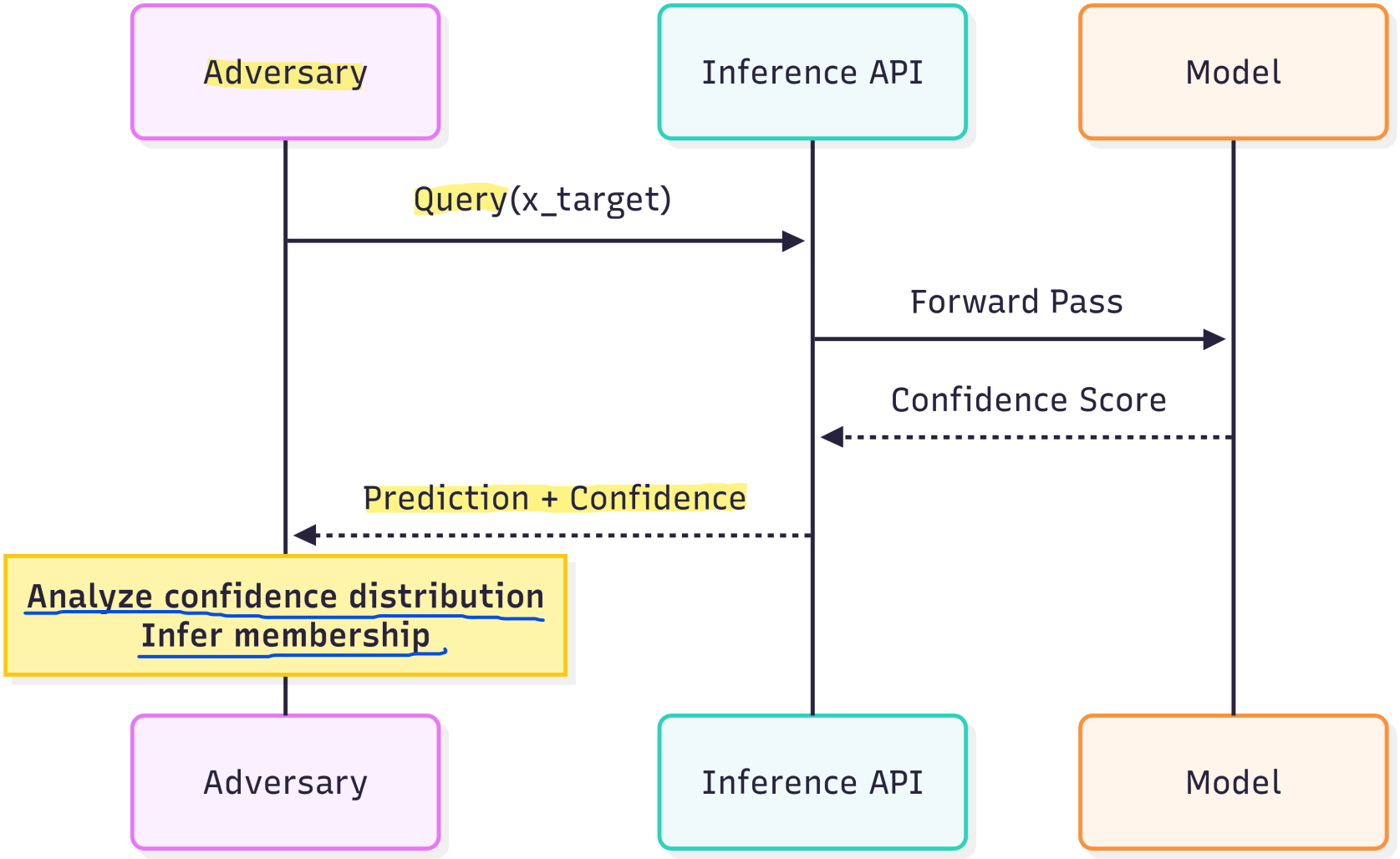
Relevant attacks:

1. Membership inference
2. Model inversion

Determining with high statistical certainty whether a specific record was included in the model's training dataset.

A privacy-breaching technique where adversaries reverse-engineer a machine learning model to extract or reconstruct sensitive data used during its training.

Privacy Threat: Membership Inference



Privacy Mitigation: Differential Privacy

Mechanism

The fundamental principle is based on the concept of indistinguishability: imagine you have a dataset D (e.g., the clinical data of patients at a hospital) and a neighboring dataset D', which contains exactly the same data as D, except for a single record. An AI model satisfies Differential Privacy if, when run on D or on D', it produces an output with an almost identical probability distribution. In simple terms, if the result of the medical analysis or the model does not change significantly whether I participate in the study or not, an attacker observing the output will never be able to tell whether my personal data was present in the system or not, neutralizing the risks of re-identification.

- **Differential privacy (DP)** is a *mathematically rigorous privacy definition* where one dataset vs its neighbor should not significantly change the output distribution
- DP is commonly described via the (ε, δ)-condition

- Epsilon controls the level of privacy: it represents the maximum multiplier of the probability difference between the two datasets. The smaller epsilon is (closer to zero), the more identical the output across the two datasets will be, ensuring very strong privacy. On the other hand, an epsilon that is too small introduces a lot of noise, reducing the model's clinical accuracy.
- Delta represents the probability that the strict privacy constraint will fail: a sort of margin of error or probability of a small information leak. In secure systems, delta must be extremely small.

Formal Guarantee

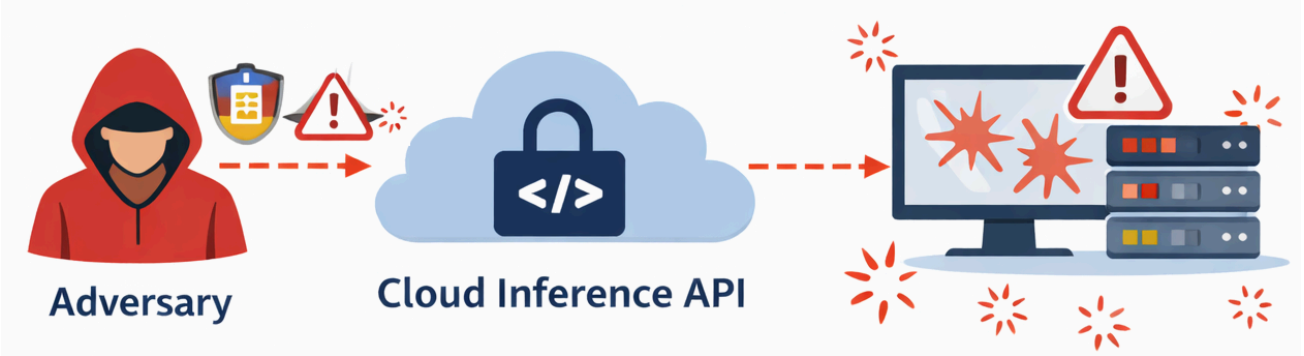
For adjacent datasets D, D' differing in one record:

$$P(M(D) \in S) \leq e^{\varepsilon} P(M(D') \in S) + \delta$$

Availability Threat: Adversarial Denial of Service

Attack Surface

Cloud inference API



Threat Description

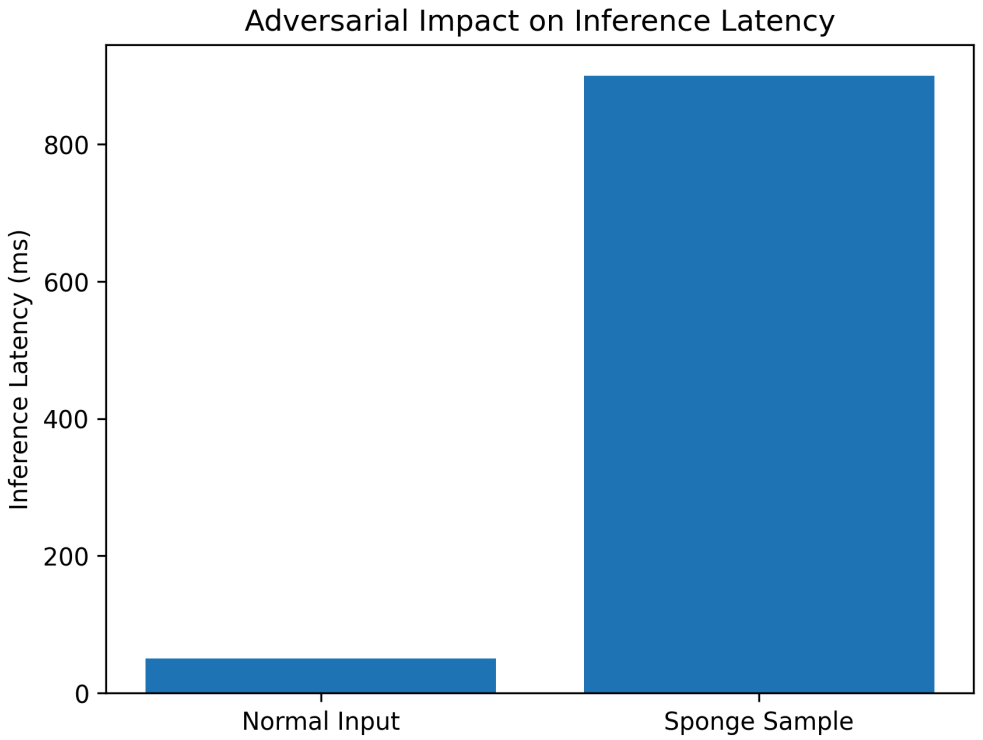
Adversarial inputs crafted to:

- Maximize computational load ("sponge samples")
- Trigger numerical instability
- Cause repeated exceptions

Availability Threat: Adversarial Denial of Service

Effect

- Increased latency
- Degraded clinical workflow refers to a breakdown in standard patient care processes
- Potential service outage



Controls

1. API rate limiting per authenticated user
2. Input validation and anomaly detection
3. Resource caps per inference request

Availability Mitigation

Measurable Metrics

- Maximum latency per request
- Rejection rate of anomalous inputs
- Service uptime percentage

Indicates the maximum time (in milliseconds) it takes for the system to respond to a single medical query. If latency spikes (e.g., from 50ms to 800ms), it means that a DoS attack or a "sponge sample" is congesting the servers.

This is the percentage of requests that the security system blocks at the entry point because they are considered strange, potentially dangerous or manipulated. A sudden spike in this metric signals an ongoing attack that the defence is neutralizing.

This is the classic availability metric (e.g., 99.9%). It measures the fraction of time during which the cloud API is active, functioning, and accessible to clinicals without interruption.

Trade-off

- False rejection of legitimate but atypical clinical cases
- Operational friction in emergency contexts

Lessons Learned (Clinical ML)

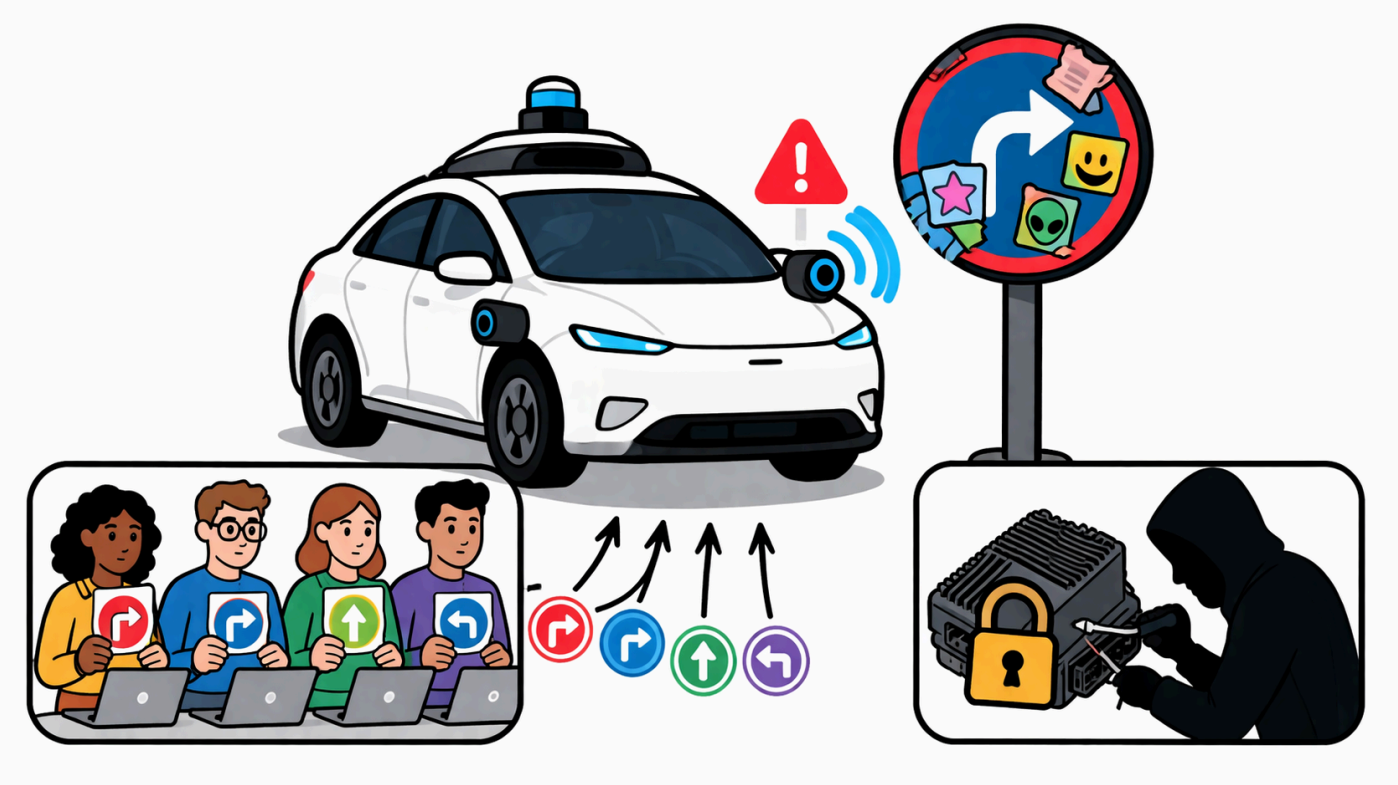
What really matters

- ML systems introduce **new attack surfaces** (training + inference)
- Security is **data-dependent**, not only system-dependent
- Privacy risks persist even without direct data access

What we need to study

- Trustworthy AI (adversarial ML, poisoning, evasion, inference attacks)
- Differential Privacy
- Federated Learning security
- Evaluation under adversarial settings

Use Case #4: Security Analysis of an Autonomous Perception System



Scenario

A perception model for an **autonomous vehicle** is deployed on an embedded ECU (Electronic Control Unit) with:

- Limited amount of ~~4 GB~~ RAM
- Limited compute budget
- Infrequent software update capability

The system must remain reliable under:

- Sensor noise and weather variability
- Physical-world adversarial manipulation (e.g., adversarial stickers)
- Supply-chain risks in the annotation pipeline

System Architecture

Function: Traffic sign recognition and scene perception

Deployment Paradigm: Edge inference + selective cloud interaction

System Architecture

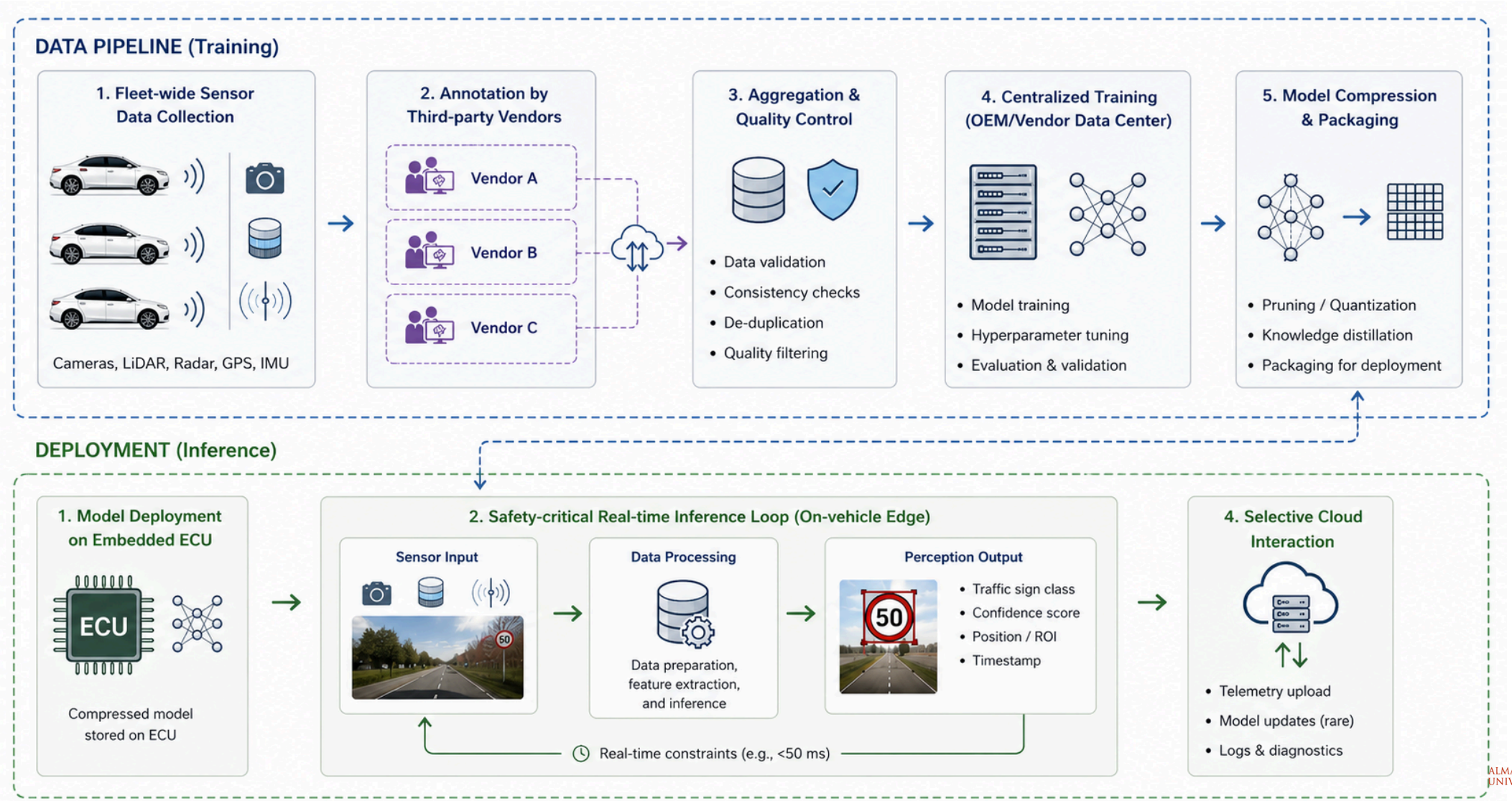
Data Pipeline

- Fleet-wide sensor data collection
- Annotation performed by three third-party vendors
- Centralized training performed in the vendor data center

Deployment

- Compressed model deployed on embedded ECU
- **No** continuous online retraining
- Safety-critical real-time inference loop

System Architecture



Threat Modeling Framework

Security objectives:

- **Integrity**: Correctness of model parameters and outputs
- **Availability**: Real-time inference under adversarial conditions
- **Authenticity**: Assurance that deployed model is untampered

Integrity Threat 1

Physical Adversarial Patches

Attack Surface

Road signs modified with localized adversarial stickers

Integrity Threat 1

Description

Threat Model

An adversary applies a perturbation δ such that:

$$\| \delta \|_\infty \leq \epsilon$$

The attacker wants the tampering to be invisible to the human eye (or to look like a harmless sticker on the sign)

but:

$$f(x) \neq f(x + \delta)$$

Even though to a human the road sign clearly remains a STOP sign, the AI becomes completely confused and misclassifies the image.

Note: $\| x \|_\infty = \max | x_i |$

Robustness-Capacity Trade-off

refers to the unavoidable engineering compromise between a machine learning model's security (robustness against adversarial attacks) and its performance.

Certified robustness typically requires:

- Robustness to perturbations $\|\delta\|_\infty \leq \epsilon$ → use of deep neural nets

  To enforce stability over perturbation regions

- Deep NNs → longer training time

  Robust training involves additional constraints and computations

- Overfitting avoidance → reduced accuracy on clean data

  Smoother decision boundaries reduce sensitivity but also precision

Integrity Threat 2

Annotation Supply-Chain Poisoning

Attack Surface

Labels provided by three external vendors

Threat Description

- Systematic label corruption
- Targeted backdoor triggers
- Class-conditional bias injection

Potential consequences

- Misclassification of specific traffic signs
- Hidden backdoor activation

Integrity Threat 2: Mitigation

Controls

1. **Redundant Annotation**
   - Each sample labeled by ≥ 3 annotators
2. **Agreement Scoring**
   - Multi-rater consistency
   - Reject samples with consistent labels below threshold

Inter-Rater Reliability (IRR) Scoring

3. **Vendor Drift Monitoring**
   - Per-vendor confusion matrix tracking
   - Statistical deviation detection

The system creates and constantly updates a confusion matrix for each individual vendor. This matrix shows how many and which types of errors that vendor makes (e.g., how many times it misinterprets a STOP sign as a speed limit). If a vendor's performance suddenly change over time in a way that deviates from the other two (creating a "statistical drift"), an alarm is triggered. For example, if Vendor A has always correctly identified 98% of signs and suddenly begins misidentifying 15% of STOP signs as "Go," the monitoring system detects a statistical anomaly and signals that that vendor's supply chain may have been compromised.

Integrity Threat 2

Trade-off

- Increased annotation cost
- Dataset size reduction
- Increased preparation latency

Availability Threat

Adversarial Sensor Inputs

Attack Surface

Camera and LiDAR (laser radar) streams

Threat Description

Inputs crafted to:

- Trigger worst-case compute paths
- Induce latency spikes
- Cause frame drops beyond control deadlines

Autonomous driving systems have strict deadlines (control deadlines, e.g., analyzing and responding to a frame within 50 ms). If the computation takes too long, the system is forced to “skip” or discard subsequent frames (frame drops) because it cannot keep up with the real-time video stream.

Consequence:

- Missed real-time constraints
- Unsafe control transitions

Availability: Mitigation

Controls

1. Input validation and preprocessing constraints
2. Worst-case latency profiling
3. Resource isolation at process level

- Before feeding the camera image or LiDAR data into the deep learning model (which is the computationally intensive part), the system performs a quick preliminary check and standardizes the data. If the input violates certain structural safety constraints, it is blocked or cleaned instantly.
- During the development phase, engineers test the software by stressing it with the worst possible inputs to map out maximum computation times. Based on this profile, safety timers (timeouts) are embedded in the car's code: if the vision algorithm is taking more than, tot time on a single frame, the calculation is forcibly truncated to prevent it from freezing the entire system.
- Many different programs run on the car's computer (brakes, steering, navigation, vision AI). This control ensures that the AI process runs in an isolated software cage, with a maximum limit on allocated RAM and computing power (e.g., up to 40% of the CPU). If the AI is attacked and reaches 100% of its allowed load, the slowdown remains confined there and does not affect the vehicle's vital processes, preventing the DoS attack from locking up the brakes or steering.

Trade-off:

- False rejection of legitimate rare scenarios

Metrics

- Maximum inference latency (ms)
- 99.9th percentile latency under adversarial load
- Frame drop rate

- The absolute maximum time recorded by the vehicle to process a single frame. It must strictly remain below the critical safety threshold (the control deadline).
- This is a crucial statistical metric. It involves sorting response times from fastest to slowest and identifying the value below which 99.9% of requests fall while the system is under attack. If even 99.9% of the frames are processed in time, it means the system is extremely stable and that the attack is failing.
- The percentage of video frames that the car is forced to discard because it cannot process them in time. The closer this percentage is to zero, the more reliable and responsive the system is.

Authenticity Threat

Model Tampering on ECU

Attack Surface

Model binary stored in on-device flash memory

Threat Description

- Unauthorized firmware modification
- Parameter replacement
- Backdoored model injection

Authenticity Mitigation: Trusted Execution

Controls

1. **Secure Boot**
    - SHA-256 integrity check of firmware at load time
2. **Trusted Execution Environment (TEE)**
    - Isolated inference runtime
    - Remote attestation

The TEE is a secure area within the main processor, completely isolated from the rest of the car's operating system.
- Isolated inference runtime: The machine learning model runs within this isolated fortress. Even if a hacker were to breach the car's infotainment system, they would still be unable to see or copy the driving model parameters inside the TEE.
- Remote attestation: This is a mechanism by which the car's chip generates undeniable cryptographic proof to demonstrate to the automaker's central servers (via the cloud) that the hardware is intact and is running only and exclusively the approved original software, without tampering.

Metrics

- Integrity verification → Indicates whether cryptographic checks have confirmed that the software and weights of the AI model are 100% identical to those released by the manufacturer, with no alterations.

Trade-off

- Increased startup latency
- Hardware cost overhead → Standard processors lack the TEEs or advanced cryptographic chips required for Secure Boot. To implement these defences, the automaker must purchase special, more expensive chips, driving up the production costs of every single vehicle.

Lessons Learned (Autonomous Systems)

What really matters

- The physical world becomes an **attack surface**
- Constraints (latency, memory) limit defenses
- Supply chain is part of the threat model

What we need to study

- Robustness of ML models
- Secure deployment (TEE, secure boot)
- Real-time constraints and worst-case analysis
- Data pipeline security

Use Case #5: Overlay Networks of Interacting AI Agents

Scenario

We consider a system composed of:

- N AI agents
- Each agent runs a Small Language Model (SLM)
- Agents interact over an overlay graph $G = (V, E)$
  - Nodes V: AI agents
  - Edges E: communication channels

The system is **fully decentralized**

- System behavior is no longer local, but **emergent and topology-dependent**

The overall behaviour of the entire system is not simply the arithmetic sum of its individual parts. On the contrary, it emerges spontaneously from the continuous and repeated interactions among the agents.

Why this is relevant

Future AI infrastructures will be:

- Distributed
- Autonomous
- Cooperative
- Heterogeneous

Examples:

- Personal agents interaction
- Distributed fraud detection
- Autonomous vehicle swarms
- Cyber threat intelligence sharing
- Edge IoT infrastructures

Why this is relevant

Agent Interaction Model

Each agent maintains a state s_i^t

- s represents the state (current information, decisions, knowledge, or beliefs) of the agent.
- i indicates the specific agent (e.g., agent number i).
- t represents the current time or interaction cycle (time step t).
In short: s_i^t is “what agent i thinks or has decided at time t”.

Update rule:

$$s_i^{t+1} = F_i\left(s_i^t, \{s_j^t : j \in \mathcal{N}(i)\}\right)$$

This equation explains how agent i changes its mind and updates its state for the next cycle (t+1). To do so, it relies on two elements fed into its reasoning function F_i:
- s_i^t (Its own prior knowledge): The agent remembers what it was thinking a moment ago.
- Information received from neighbours: This is where the famous Overlay Graph comes into play. The symbol N(i) represents the neighbourhood, i.e., the list of agent i’s direct neighbours based on the graph’s edges. The agent collects the states (s_j^t) of all its neighbours j.

where:

- $\mathcal{N}(i)$ = neighbors of agent i
- F_i implemented via SLM reasoning → This is the reasoning process of the SLM. The agent collects data from its neighbours, inputs it into a prompt, and the language model “reflects” on it and generates the new decision for time t+1.

Security implication

Local compromise → Global instability

- Local infection: Suppose an attacker manipulates the input of a single agent or poisons its model. Agent 1’s state at time t (s_1^t) becomes corrupted, incorrect, or malicious.
- The domino effect: At the next cycle (t+1), all of Agent 1’s neighbours (the nodes belonging to N (1)) will read its corrupted state and use it within their reasoning function F_i.
- Global instability: Since SLMs tend to trust and process the information they receive from neighbouring nodes in the overlay, the corrupted state will act as an “algorithmic virus.” Cycle after cycle (t+1, t+2, ...), the error will propagate from neighbourhood to neighbourhood, causing the entire decentralized network to converge toward a catastrophic decision (global instability).

Threat 1: Compromised High-Centrality Agent

Assume node k has:

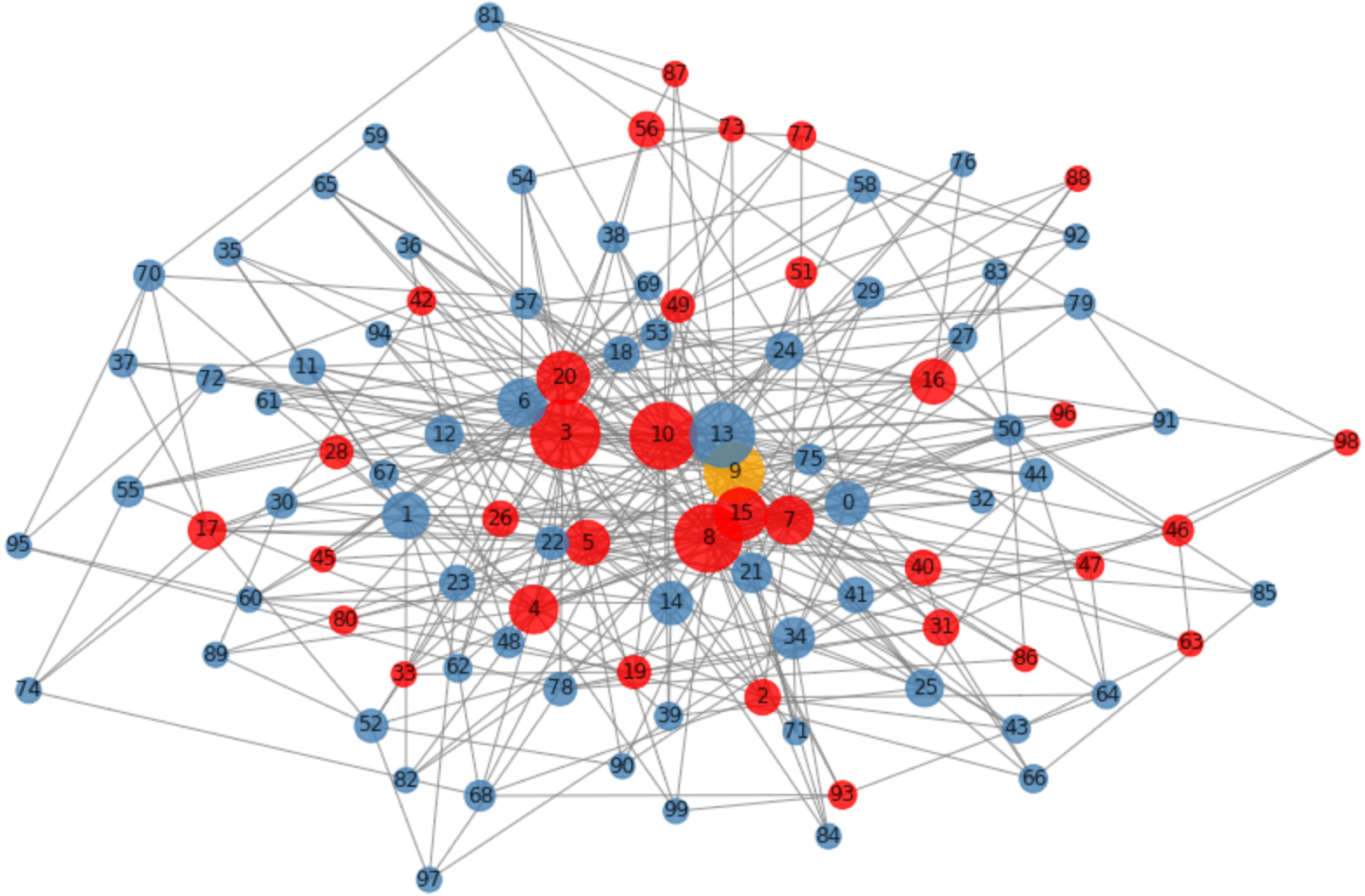
- High degree centrality — It is the number of direct arcs connected to the node. The higher the degree, the more direct neighbors the node has.
- High betweenness centrality — It measures how many times a node is located on the “shortest path” connecting any two nodes in the network. It is the mandatory gateway. This agent acts as a traffic controller for information. If node A (distant) needs to send information to node B (distant) or coordinate with it, the message will almost certainly have to pass through this central node and be processed by it.
- High eigenvector centrality — It measures how important and central your neighbours are. You have high eigenvector centrality if you are connected to nodes that are themselves highly influential.

If compromised: $s_k^t \rightarrow s_k^t + \delta$

Impact

- Rapid misinformation propagation
- Cascading behavioral drift
- System-wide instability

Threat 1: Compromised High-Centrality Agent



Threat 2: Coordinated Minority Attack

Subset $S \subset V$ behaves adversarially

Effects:

- Biased consensus
- Polarization

If one part of the network resists the attack while another part is influenced by the subset S, the graph splits into two or more isolated ideological/algorithmic factions that can no longer find common ground. The healthy agents cease to cooperate uniformly.

- Decision boundary shift

Critical question:

Is there a **phase transition threshold** for successful attack?

When applied to the security of this network, this raises the question: what is the minimum fraction of corrupted nodes (|S|/|V|) required to bring down the system?
- Below the threshold: If the attacker controls only a few nodes (e.g., 5% of the network), the structure of the overlay graph and the defence mechanisms of the healthy nodes manage to " absorb" the malicious noise. The attack fails, and the network still converges to the truth.
- Above the threshold: If the attacker exceeds the critical threshold (which, depending on the graph topology, often hovers around precise values such as 20% or 33% of compromised nodes), the system suffers an instantaneous structural collapse. The network's self-healing capacity is nullified, and the effects of biased consensus or polarization inevitably prevail.

Threat 3: Topology-Driven Amplification

Certain graph structures amplify perturbations:

- Scale-free networks
- Highly clustered communities

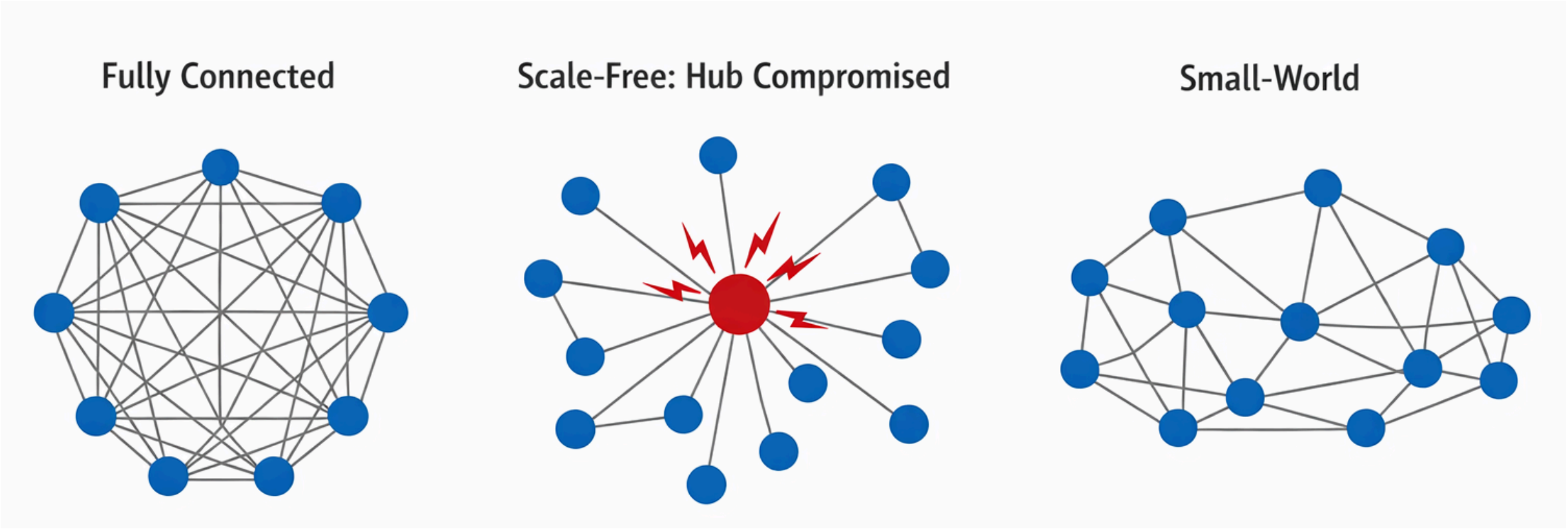
These are highly unbalanced networks, where the vast majority of nodes have very few connections, while a very small number of nodes (called hubs) have thousands of connections. The internet, social networks (e.g., Twitter/X, where a few celebrities have millions of followers) are examples of scale-free networks.
Why they amplify perturbations: if the attacker targets a peripheral node, the attack dies immediately. But if the attacker targets a Hub (a highly central node), that single manipulated input is instantly propagated across a massive portion of the network in a single time step (t+1). The Hub amplifies the attack, causing it to bypass every local defense.

Imagine closed groups or cliques of AI agents that communicate extensively with each other but very little with the outside world. Within a highly clustered community, if Node A communicates with B and B communicates with C, it is highly likely that A also communicates directly with C.
Why they amplify perturbations: These structures create deadly echo chambers. If an attacker injects a lie or a corrupted state into this clique, the malicious information will begin to bounce continuously among the group members. Since the update rule forces models to reprocess data from neighbours, the nodes will continue to confirm the lie to one another. This continuous self-poisoning causes a rapid Decision Boundary Shift within the community, polarizing it and rendering it completely deaf to the truth coming from the rest of the network.

Small perturbations may propagate exponentially

Security is topology-sensitive

Threat 3: Topology-Driven Amplification



Why Network Theory is Necessary

This is not a classical ML security problem, we are deadling with a **dynamical system over a graph**

We must analyze:

- Degree distribution
- Network connectivity
- Clustering coefficient
- Percolation thresholds

Degree distribution: this tells us how connections are distributed across the network. Is there a democratic balance (where all nodes have about 4 or 5 neighbors), or are we dealing with a scale-free network (where 99% of nodes have only one connection and there are 2 or 3 dominant super-hubs)? Understanding this map allows us to immediately identify the most valuable targets to protect.

Network connectivity: measures how connected the network is and how many alternative paths exist for information to travel. High connectivity makes the network robust if a node goes offline (because messages can take the long way around), but if the system is under attack, an overly connected network allows the algorithmic virus to reach every corner of the graph in just a few steps.

Clustering coefficient: calculates the density of local “cliques” or closed communities. As we saw in the previous slide, a high clustering coefficient indicates the risk of creating echo chambers where poisoned nodes reinforce each other’s views, accelerating decision boundary shifts and fracturing the network (polarization).

Percolation thresholds: calculating it means mathematically determining how many nodes the attacker must compromise before the infection becomes unstoppable and triggers the phase transition toward the system’s global collapse.

Mitigation 1: Trust-Weighted Aggregation

Modify update rule:

$$s_i^{t+1} = F_i\left(s_i^t, \sum_{j \in \mathcal{N}(i)} w_{ij} s_j^t\right)$$

Where:

- w_{ij} = trust weights
- dynamically updated

Goal: reduce adversarial influence

Mitigation 2: Topology Engineering

Design graph to improve robustness:

- Increase net connectivity
- Limit hub dominance
- Avoid extreme scale-free structures
- Introduce redundancy

In this scenario, security is partially a graph design problem

Mitigation 3: Byzantine-Resilient Consensus

Use robust aggregation:

- Median-based update — Instead of calculating the mean, the AI agent takes all the neighbours' responses, sorts them in order from smallest to largest, and selects the value that falls exactly in the middle (the median). The erroneous inputs from the Byzantine nodes end up at the ends of the array and are completely ignored by the calculation. The impact of the malicious nodes is nullified.
- Trimmed mean — Before calculating the mean, the system discards a fixed percentage x% of the smallest values and x% of the largest values received from the neighbourhood. The standard arithmetic mean is calculated only on the remaining values (the central and stable ones).
- Outlier rejection — The AI agent analyses the geometric distribution of all received messages. It calculates where the bulk of the data is concentrated and, using distance metrics (such as standard deviation or Euclidean distance), identifies values that deviate significantly from the group (known as outliers) and reject them.

Goal: ~~Bound~~ adversarial impact is limited even if fraction f of nodes is malicious

Mitigation 4: Influence Monitoring

Compute periodically:

- Node centrality metrics
- Influence functions
- Local divergence indicators

Node centrality metrics: As we saw earlier (Degree, Betweenness, Eigenvector), the system constantly monitors the power map. If a node suddenly gains too many connections or becomes a mandatory bridge for information, the system flags it as a “High-Value Target” and increases the level of surveillance on it, because it knows that its compromise would be catastrophic.

Influence functions: The influence function mathematically calculates how much Node A’s opinion weighs on Node B’s final decision. If the system detects that the decisions of an entire neighbourhood of healthy nodes depend excessively and disproportionately on inputs sent by a single specific node, that node becomes a suspect under special surveillance.

Local divergence indicators: These measure how much an agent’s state is deviating (diverging) from the historical or logical average of its area of the graph. If a node begins to exhibit an unusual response trajectory relative to the standard behaviour expected for its topological position, it means that something is destabilizing it.

Early detection of:

- Behavioral drift
- Coordinated anomalies

By monitoring the influence, the system detects that the decision-making boundaries of the SLMs are shifting abnormally and sounds the alarm before the algorithmic brainwashing is complete.

Lessons Learned (AI Agent Networks)

What really matters

- Security is **emergent and topology-dependent**
- Local compromises can cause **global failures**
- Trust must be **distributed and dynamic** →

- Distributed: Information is not accepted based on the word of a single node; instead, robust aggregation primitives are used, forcing the network to reach a distributed and democratic consensus that excludes adversarial extremes.
- Dynamic: The system must constantly monitor centrality metrics and influence functions to detect whether a node is undergoing behavioural drift. If an indicator triggers, the network must be ready to instantly recalibrate trust in that node, isolating it or reshaping the graph topology before it is too late.

What we need to study

- Network theory and graph robustness
- Distributed consensus under adversaries
- Trust and reputation systems
- Dynamical systems and stability analysis

Part III: Some problems and techniques we have identified so far

Foundational Terms - CIA

Foundational Terms - CIA + ML Extension

The classic **CIA triad** remains the backbone:

- When an attack compromises the model's integrity, it still works perfectly well 99% of the time. There is no general collapse in performance. The attacker simply wants the model to make a specific, incorrect decision at a precise moment. Example (self-driving car): The car is driving perfectly, recognizing pedestrians, lanes, and traffic lights, but when it encounters the specific STOP sign with the adversarial sticker, it reads it as a 50 km/h speed limit.
- When an attack targets Availability, the goal is to prevent the system from doing its job, rendering it unusable, extremely slow, or forcing it to shut down (Denial of Service). Example (Sponge Attack / Latency): The attacker sends images designed to cause the GPU's calculations to loop (as seen in worst-case latency profiling). Instead of processing 30 frames per second, the car drops to 2 frames per second (skyrocketing frame drop rate). The vision system is unavailable because it is too slow to ensure safe driving.

Property	Classic Definition	ML Extension
Confidentiality	Data visible only to authorized parties	Training data, model weights, inference outputs visible only to authorized parties
Integrity	Data/system not modified without authorization	Model predictions not manipulated by adversary Ensure that the output is genuine and has not been tampered with by an attacker. If a malicious user provides misleading input (an adversarial attack) to deliberately cause the model to make a mistake, they are violating the integrity of the AI system.
Availability	System accessible when needed	Model not degraded by denial-of-service or poisoning AI must not stop functioning or experience a decline in performance due to attacks that overwhelm computational resources or that corrupt the data (poisoning), rendering the model useless or unreliable for legitimate users.

Threat Actors & Their Capabilities

We classify adversaries along two axes:

Knowledge:

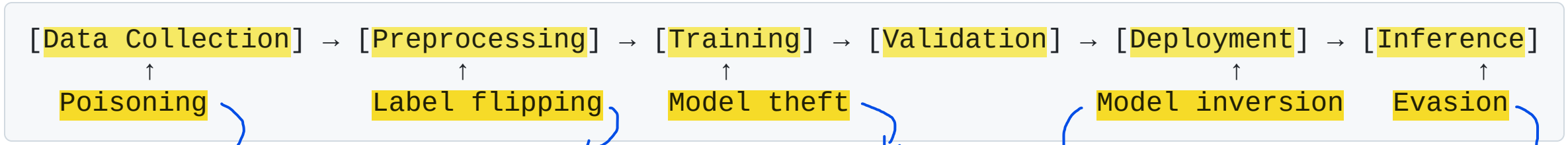
- **White-box**: full access to model architecture, weights, training data
- **Gray-box**: partial knowledge (architecture known, weights unknown)
- **Black-box**: query-only access (most realistic in production)

Goals:

- **Integrity attacks**: cause misclassification
- **Privacy attacks**: extract sensitive information from model
- **Availability attacks**: degrade model performance
- **Backdoor attacks**: embed hidden behavior triggered by a specific input pattern

Threat Modeling for ML Systems

The ML pipeline as an attack surface:



What the attacker does: The attacker intervenes at the very earliest stage by introducing malicious, falsified, or altered data directly into the dataset that will be used for training.

Objective and Impact: The attacker aims to corrupt the model's future behavior. If the attacker injects "poisoned" data (e.g., malware images disguised as legitimate files), the model will learn an incorrect rule, compromising its future integrity (add trigger to training data) or availability (degrade performance).

What the attacker does: During the data cleaning and preparation phase, the attacker manages to swap the labels in a targeted manner.

Objective and Impact: This is a specific and very powerful form of poisoning. If the attacker systematically reverses the label of 5% of the STOP sign images, the model will become confused during training, creating a dangerous security flaw (backdoor) at runtime.

What the attacker does: The attacker targets the phase during which the model is calculating its internal parameters (the weights) or the infrastructure where it resides. The theft can occur by breaching the training servers or, more commonly, by repeatedly querying the model (a black-box attack) to map its responses and train a "clone" model at no cost.

Objective and Impact: Directly violates confidentiality. The attacker steals intellectual property (the model weights), nullifying the company's competitive advantage without having to spend resources on GPUs and data collection.

What the attacker does: When the model is packaged and deployed to the production infrastructure, the attacker exploits the APIs or access to the model to perform a reverse mathematical process: starting from the AI's outputs, they work backward to reconstruct the original inputs.

Objective and Impact: This is a very serious attack on the confidentiality of sensitive data (training data). For example, if the model is a medical AI, an attacker can use model inversion to reconstruct the clinical images or medical records of the real patients used for training.

What the attacker does: This vulnerability manifests during the final phase (Inference), that is, when the model is active and responds in real time to real-world inputs. The attacker applies a targeted and often imperceptible perturbation to the input (e.g., the pixels of a road sign or the code of a virus) to deceive the model during computation.

Objective and Impact: It compromises the integrity of predictions. The model itself is not modified (the weights remain intact), but the manipulated input forces the AI to "evade" the correct classification (e.g., mistaking a STOP sign for a speed limit or failing to detect malware), allowing the attacker to bypass control systems.

AI for Cybersecurity

to conclude ... since we gave it for granted

AI as a Tool for Defense Cybersecurity

Application	ML Approach	Key Challenge
Intrusion Detection	Anomaly detection, sequence models (e.g., LSTM) — Anomaly detection is performed by analysing traffic as a time series using sequential models such as LSTMs	Maintaining low false positives under high-volume traffic — With billions of data packets per day, the AI must maintain a false positive rate close to zero to prevent overwhelming security teams (alert fatigue).
Malware Classification	Static/dynamic analysis, GNNs — and GraphNNs for mapping the code execution flow as a graph of function calls.	Robustness to adversarial evasion techniques — Attackers modify files using obfuscation techniques or by injecting useless code to evade the neural network's detection without disrupting the malware's functionality.
Fraud Detection	Supervised learning (e.g., gradient boosting), online learning — Fast supervised learning algorithms such as Gradient Boosting (e.g., XGBoost), combined with online learning to update the model in real time.	Drift due to evolving attacker adaptation — Scammers are constantly changing their payment strategies, causing data drift that quickly renders old models obsolete.
Behavioral Analytics	Sequence modeling, user embeddings — Sequential models and user embeddings (mathematical representation of behavior) to track employee usage patterns and detect credential theft.	Preserving user privacy while modeling behavior — To be able to identify anomalies while complying with data protection regulations (GDPR) and preventing the leakage of sensitive employee data.
Threat Intelligence	NLP on CTI reports, knowledge graphs — NLP (Natural Language Processing) techniques for reading CTI (Cyber Threat Intelligence) text reports and automatically organizing them into knowledge graphs.	Data quality and noise in OSINT — Information from OSINT and forums on the Deep Web is fragmented, incomplete, and riddled with noise or deliberate attempts at disinformation.

Cyber threat intelligence (CTI) is the collection, analysis, and processing of data to understand the motives, targets, and attack behaviors of cybercriminals. It transforms raw security data into actionable knowledge, allowing organizations to anticipate threats and proactively defend their systems rather than simply reacting to attacks after they occur.

Some Recommended Resources to Start

Primary references:

- Goodfellow et al. — *Deep Learning* (MIT Press) — Ch. on Robustness
- Dwork & Roth — *The Algorithmic Foundations of Differential Privacy* (2014)
- Papernot et al. — *SoK: Security and Privacy in ML* (IEEE EuroS&P 2018)

Tools & frameworks:

- `CleverHans`, `ART (IBM)` — adversarial attack/defense libraries
- `Opacus` (PyTorch), `TF Privacy` — differential privacy
- `PySyft` — federated learning & SMPC
- `FATE` — industrial federated learning framework

Prof. Stefano Ferretti

Department of Computer Science and Engineering

University of Bologna

s.ferretti@unibo.it